



计算机视觉表征与识别

Chapter 6: Edges and Boundaries

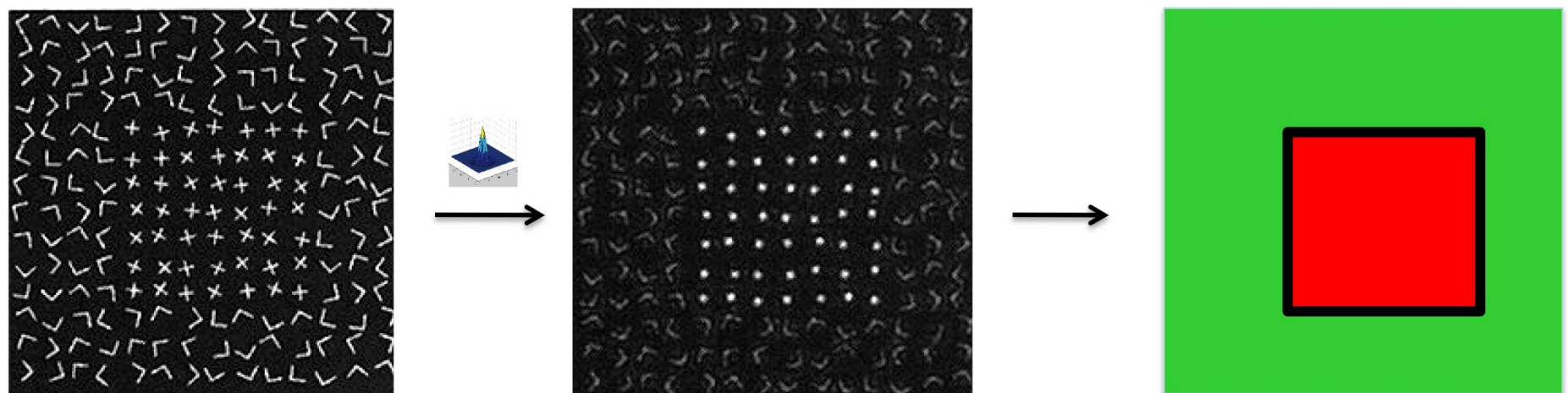
王利民

媒体计算课题组

<http://mcg.nju.edu.cn/>



From texture to edge/segmentation

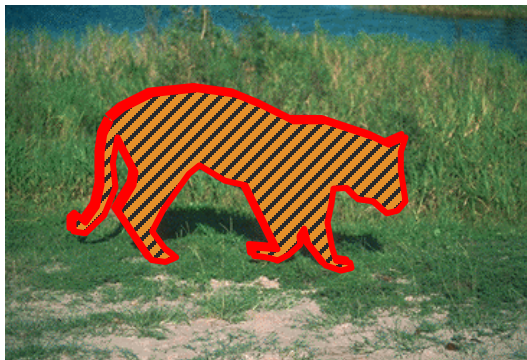
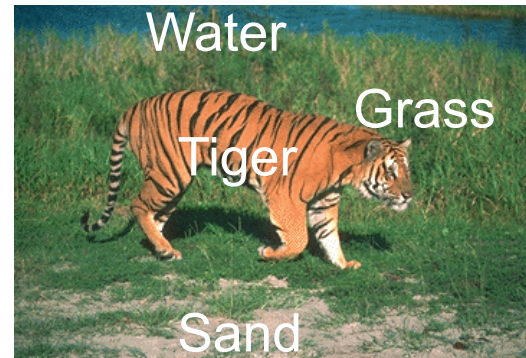


Low level Vision

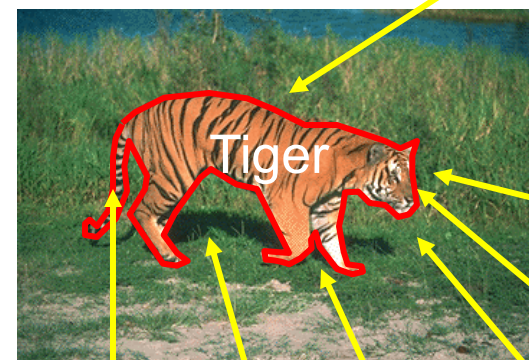
Middle level Vision



From Pixels to Perception



**Mid-level operations of
Segmentation and Grouping**



back

head

eye

mouth

tail

legs

shadow



Figure / Ground

Finding groups of pixels that go together
(parts, objects, textures, holes)



Predicted scene categories[■]:

forest - broadleaf (0.498), swimming hole (0.402), bayou (0.062)



Predicted scene categories[■]:

forest - broadleaf (0.979)



Figure / Ground

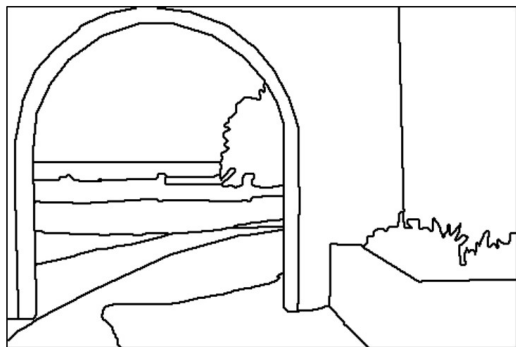
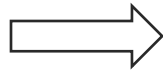
Finding groups of pixels that go together
(parts, objects, textures, holes)



<http://twistedifter.com/2015/03/mind-bending-optical-illusion-paintings-by-rob-gonsalves/>



Regions

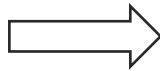
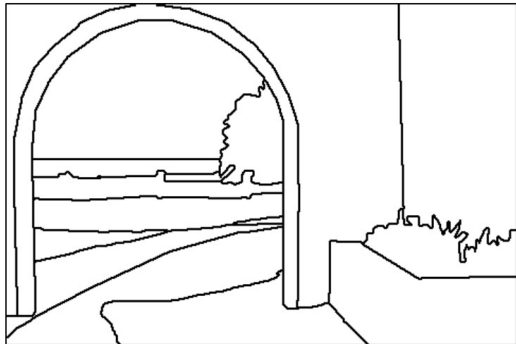


Decomposing image into K connected regions
(Clustering task)

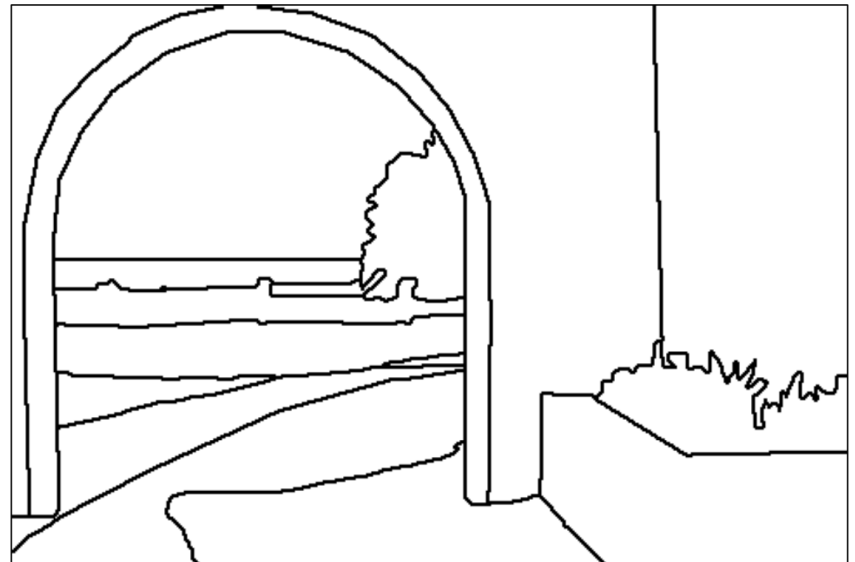




Edges/Contour



$H \times W \times \{0,1\}$ classification problem





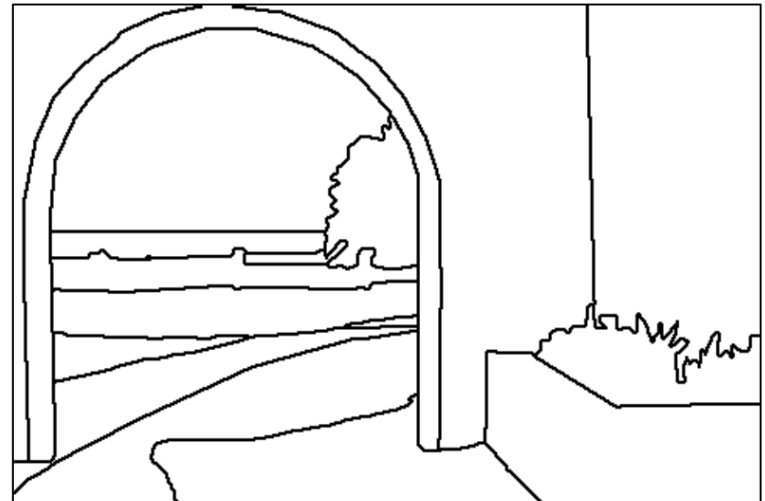
Are the tasks equivalent



Segmentation



Boundaries

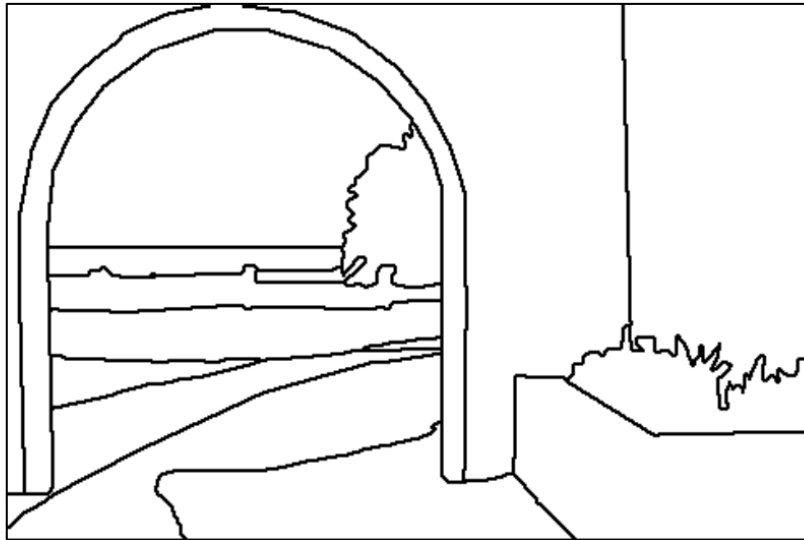




Are the tasks equivalent



Boundaries



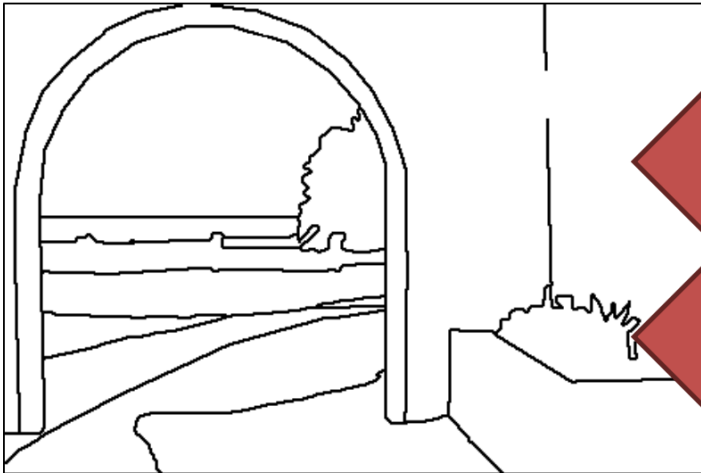
Segmentation



Are the tasks equivalent



Boundaries



Segmentation



Contours have to be closed!



Today's class



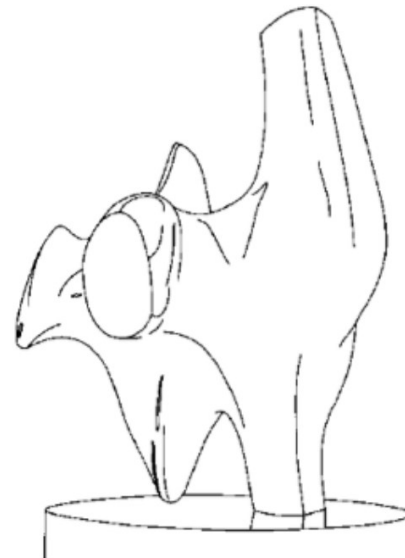
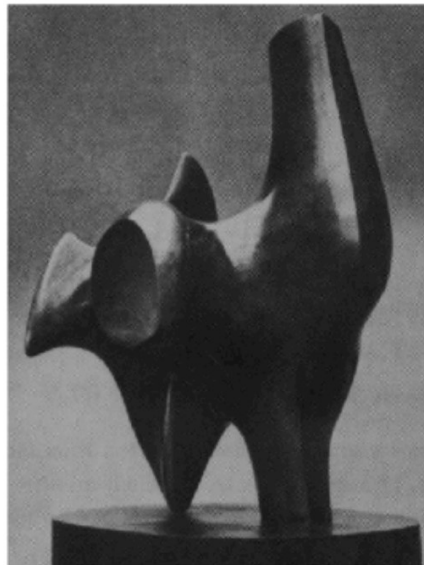
- **Introduction to edge detection**
- Gradients and edges
- Canny edge detector
- Object contour
- Pb edge detector
- Straight line detection



First problem: edge detection



- Goal: compute something like a line drawing of a scene





Edge detection



- **Goal:** map image from 2d array of pixels to a set of curves or line segments or contours.

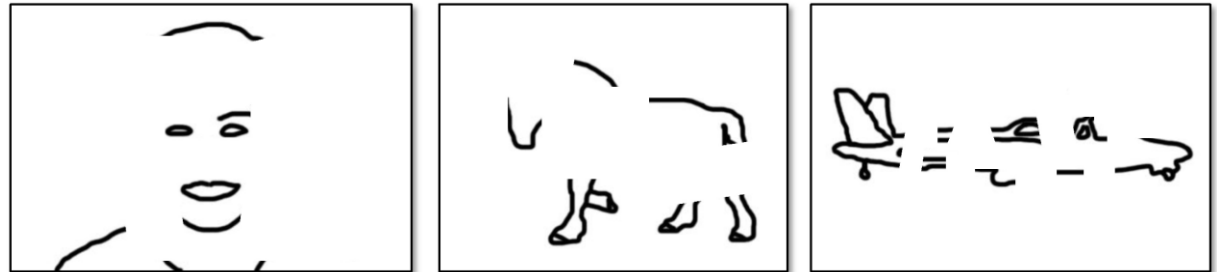


Figure from J. Shotton et al., PAMI 2007

- **Main idea:** look for strong gradients, post-process



Why



- Edges reflect intrinsic properties of a scene
 - Capture shape information
 - Independent of illumination
- Our human visual system does something like this
- Good initial step for solving other problems
 - Recognition, tracking, etc.



Issues



- No precise problem formulation
- Much harder than it seems to be
- **Edge detectors** usually work by detecting “big changes” in image intensity
- **Boundary** is contour in the image plane that represents a change in pixel ownership from object or surface to another.

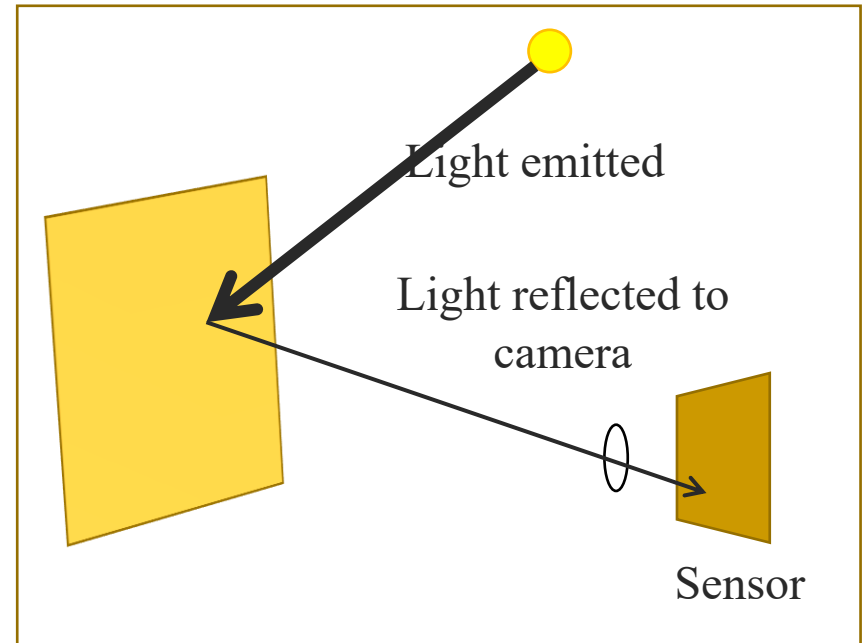


Recall: How much light is recorded



■ Major factors

- Illumination strength and direction
- Surface geometry
- Surface material
- Nearby surfaces
- Camera gain/exposure



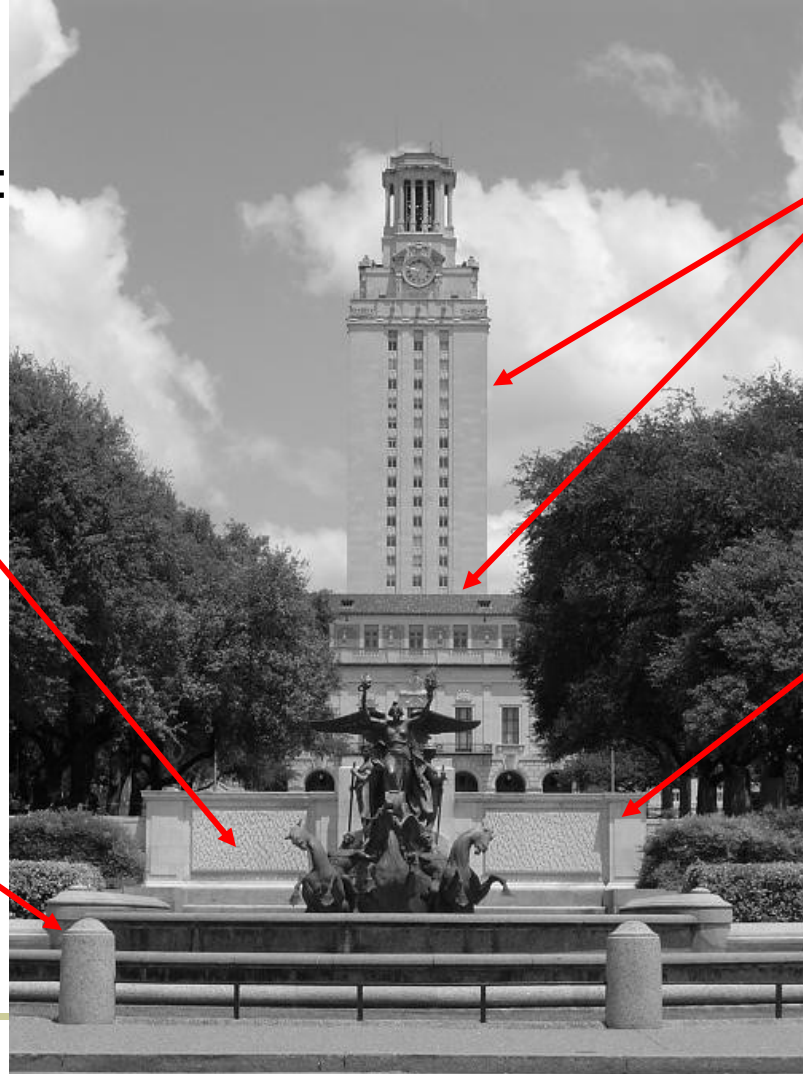


What causes an edge?



Reflectance change:
appearance
information, texture

Change in surface
orientation: shape



Depth discontinuity:
object boundary

Cast shadows

Slide credit:
Kristen Grauman



Today's class



- Introduction to edge detection
- **Gradients and edges**
- Canny edge detector
- Object contour
- Pb edge detector
- Straight line detection

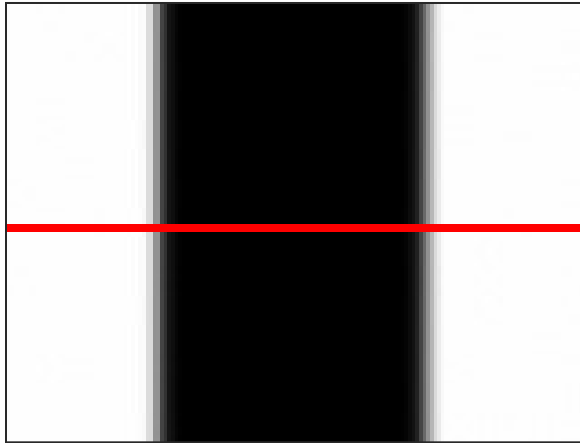


Derivatives and edges

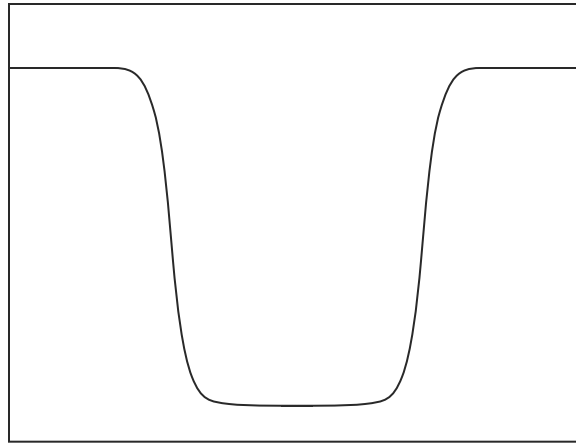


An edge is a place of rapid change in the image intensity function.

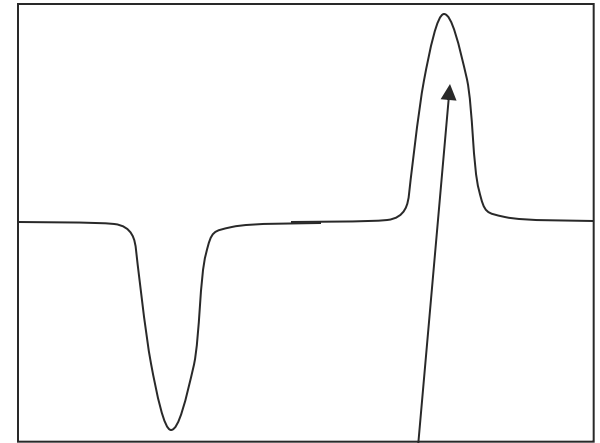
image



intensity function
(along horizontal scanline)



first derivative



edges correspond to
extrema of derivative



Derivatives with convolution



For 2D function, $f(x,y)$, the partial derivative is:

$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

For discrete data, we can approximate using finite differences:

$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$

To implement above as convolution, what would be the associated filter?

Slide credit: Kristen Grauman

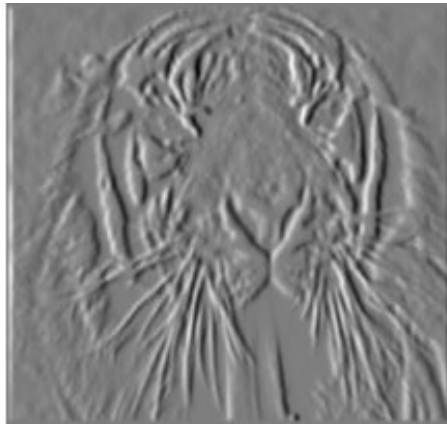


Partial derivatives of an image



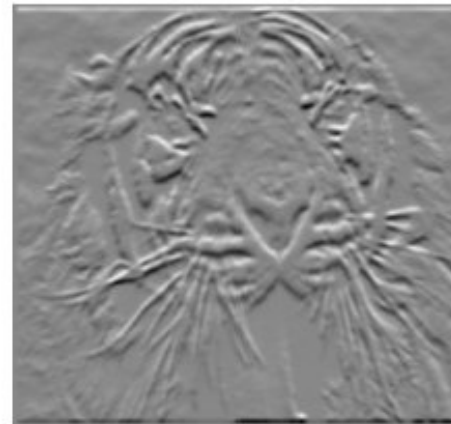
$$\frac{\partial f(x, y)}{\partial x}$$

-1	1
----	---



$$\frac{\partial f(x, y)}{\partial y}$$

-1	?	1
1	or	-1



Which shows changes with respect to x?

(showing filters for correlation)



Finite difference filters



- Other approximations of derivative filters exist:

Prewitt: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} ; M_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$

Sobel: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} ; M_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$

Roberts: $M_x = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} ; M_y = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Source: K. Grauman



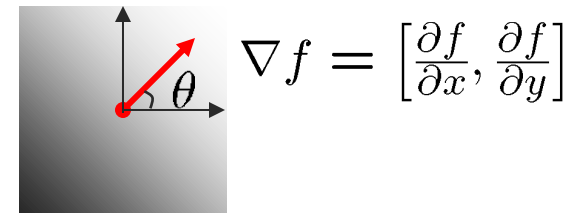
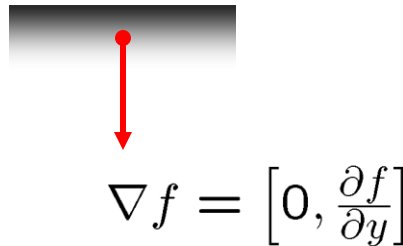
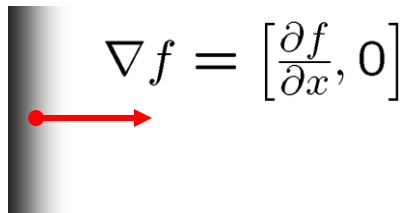
Image gradient



The gradient of an image:

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

The gradient points in the direction of most rapid change in intensity

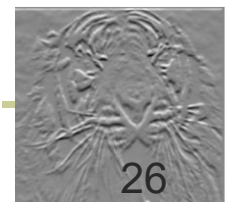


The **gradient direction** (orientation of edge normal) is given by:

$$\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$$

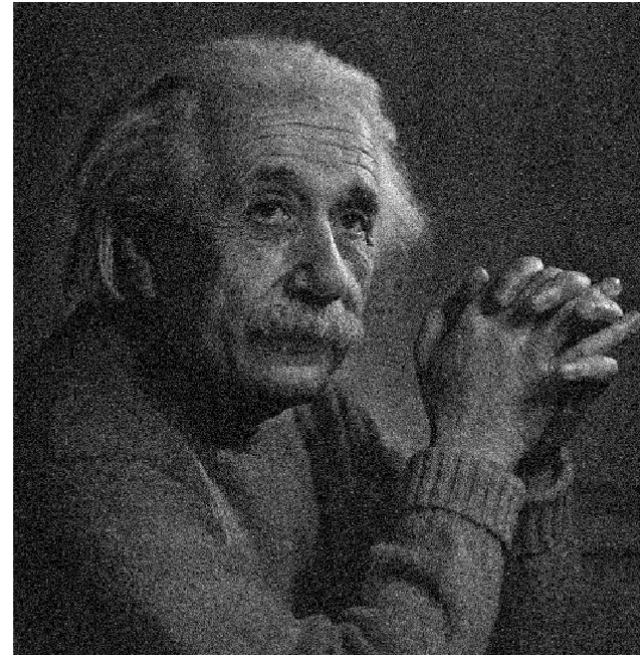
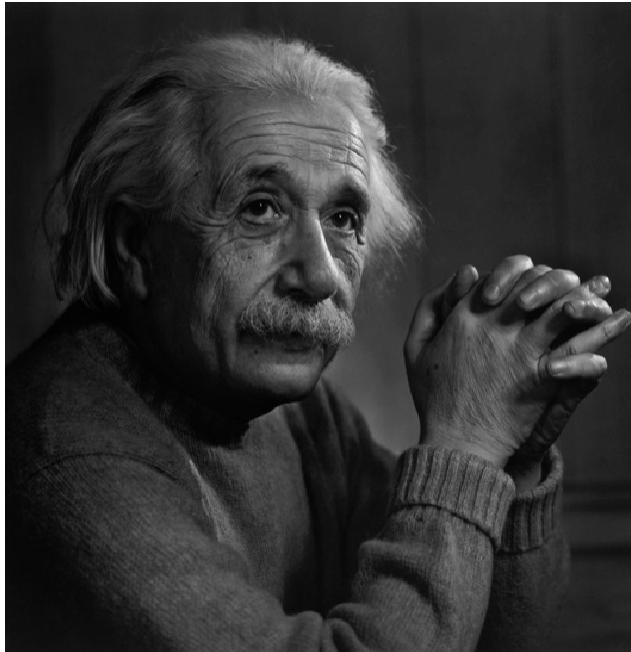
The **edge strength** is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$





Effects of noise





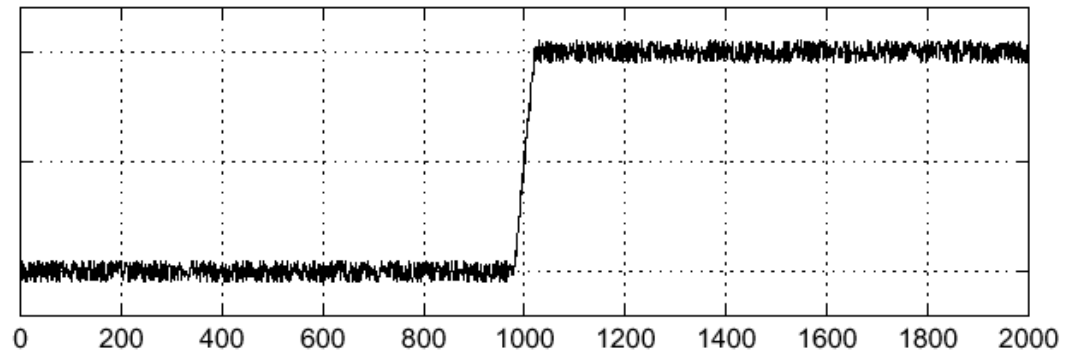
Effects of noise



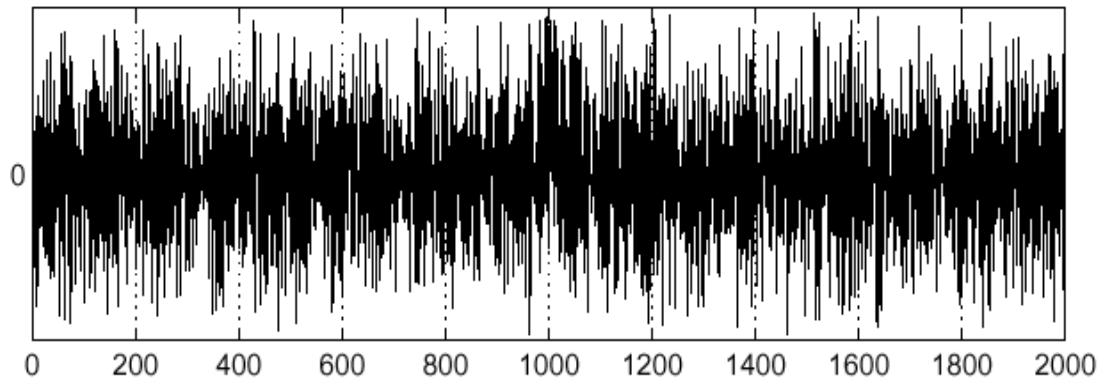
Consider a single row or column of the image

- Plotting intensity as a function of position gives a signal

$$f(x)$$



$$\frac{d}{dx}f(x)$$



Where is the edge?



Effects of noise



- Difference filters respond strongly to noise
 - Image noise results in pixels that look very different from their neighbors
 - Generally, the larger the noise the stronger the response
- What can we do about it?

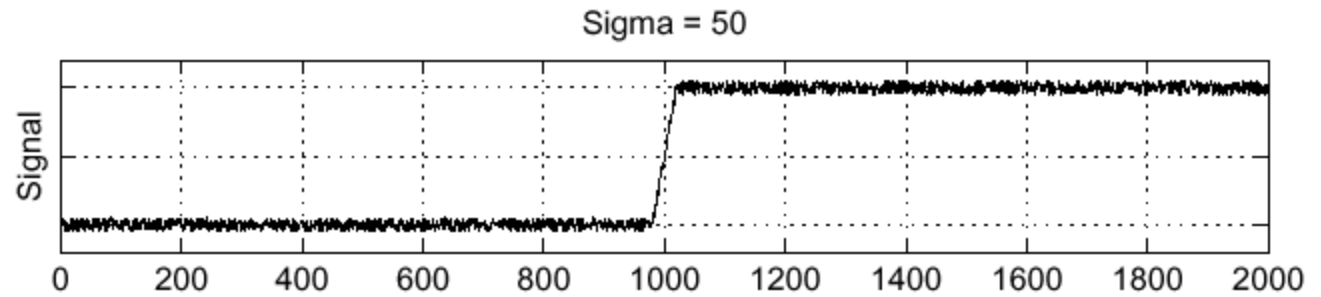
Source: D. Forsyth



Solution: smooth first



f



h



Where is the edge?

Look for peaks in $\frac{\partial}{\partial x}(h \star f)$



Derivative theorem of convolution

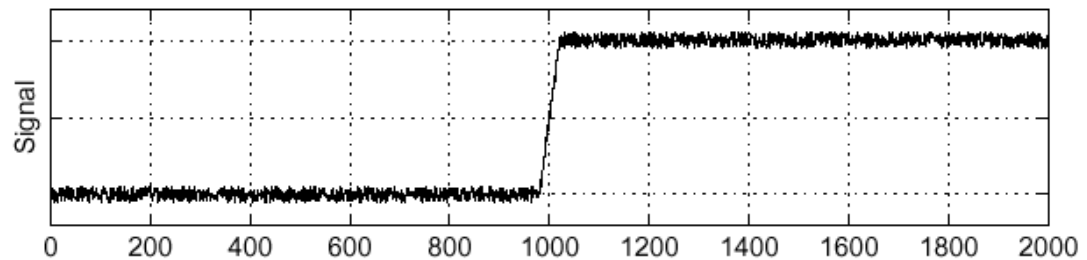


$$\frac{\partial}{\partial x}(h \star f) = \left(\frac{\partial}{\partial x}h\right) \star f$$

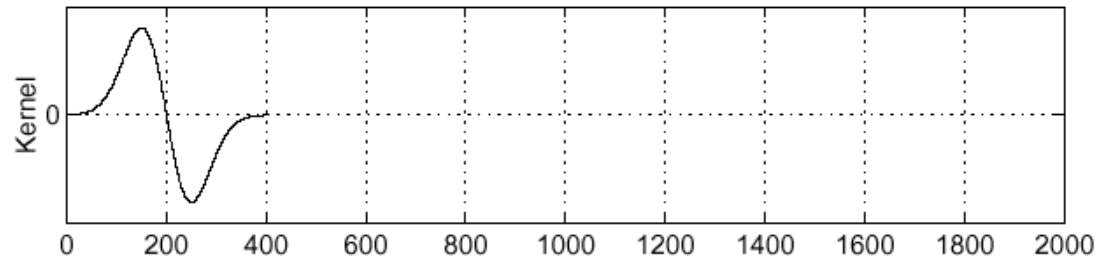
Differentiation property of convolution.

Sigma = 50

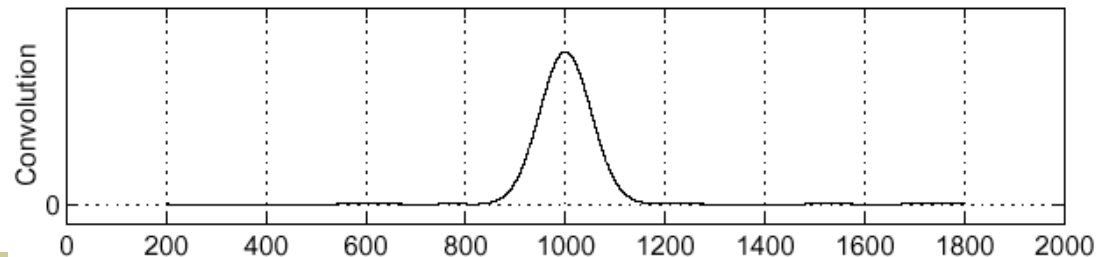
f



$\frac{\partial}{\partial x}h$



$\left(\frac{\partial}{\partial x}h\right) \star f$



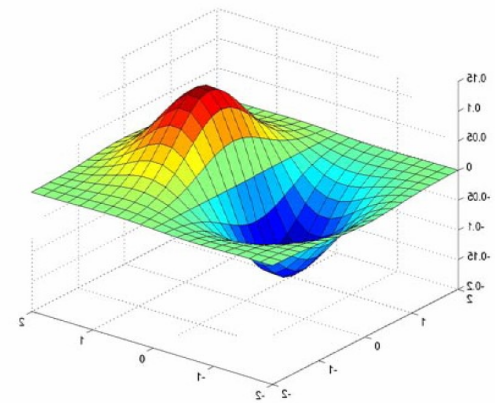


Derivative of Gaussian filters



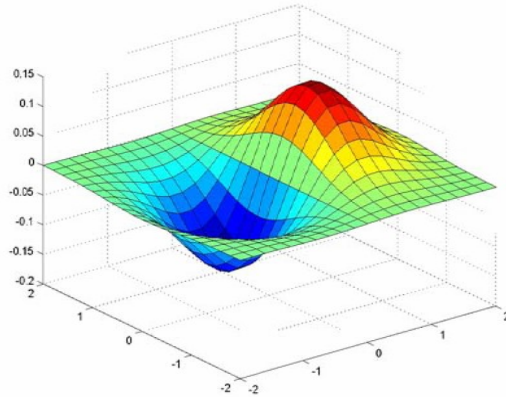
$$(I \otimes g) \otimes h = I \otimes (g \otimes h)$$

$$\begin{bmatrix} 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0219 & 0.0983 & 0.1621 & 0.0983 & 0.0219 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \end{bmatrix} \otimes \begin{bmatrix} 1 & -1 \end{bmatrix}$$

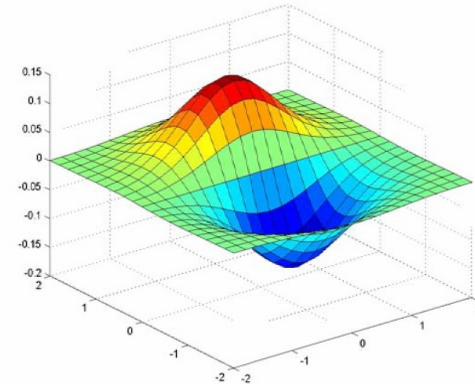
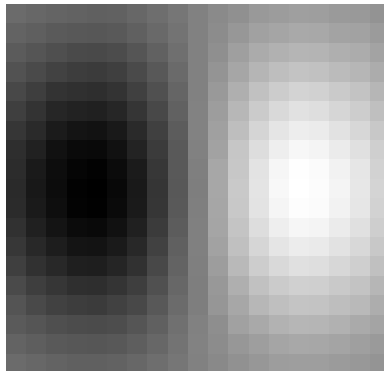




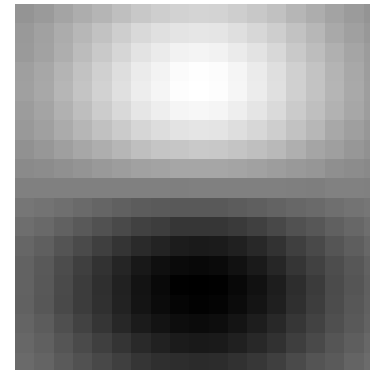
Derivative of Gaussian filters



x-direction



y-direction





The Sobel Operator: A common approximation of derivative of gaussian



- Common approximation of derivative of Gaussian

$$\frac{1}{8} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

s_x

$$\frac{1}{8} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

s_y

- The standard defn. of the Sobel operator omits the $1/8$ term
 - doesn't make a difference for edge detection
 - the $1/8$ term is needed to get the right gradient value



Mask properties



■ Smoothing

- Values positive
- Sum to 1 $\rightarrow$ constant regions same as input
- Amount of smoothing proportional to mask size
- Remove “high-frequency” components; “low-pass” filter

■ Derivatives

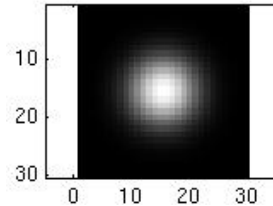
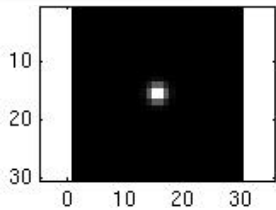
- _____ signs used to get high response in regions of high contrast
- Sum to ____ $\rightarrow$ no response in constant regions
- High absolute value at points of high contrast



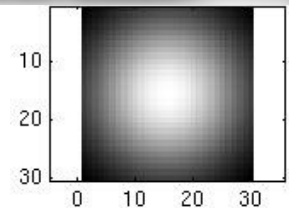
Smoothing with a Gaussian



Recall: parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel, and controls the amount of smoothing.

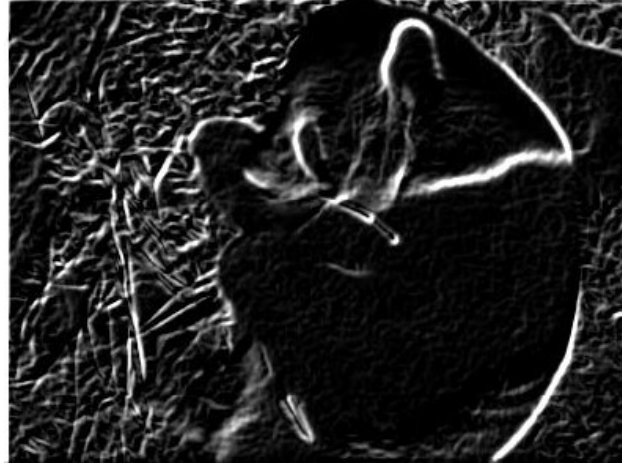


...

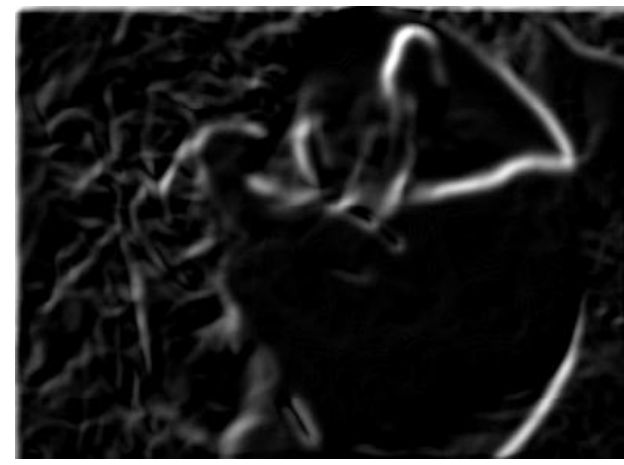




Effect of σ on derivatives



$\sigma = 1$ pixel



$\sigma = 3$ pixels

The apparent structures differ depending on Gaussian's scale parameter.

Larger values: larger scale edges detected

Smaller values: finer features detected



Gradients -> edges



Primary edge detection steps:

1. Smoothing: suppress noise
2. Edge enhancement: filter for contrast
3. Edge localization

Determine which local maxima from filter output are actually edges vs. noise

Thresholding and thinning

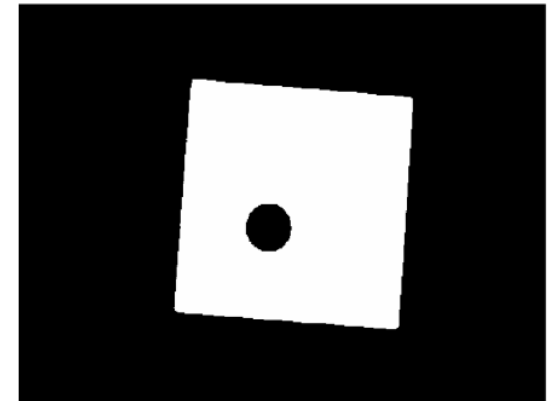
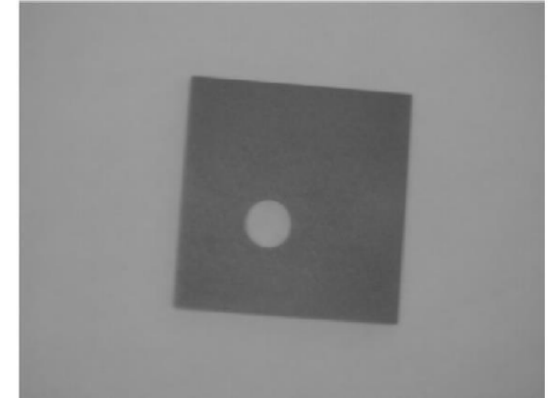
- ⇒ Larger values: larger-scale edges detected
- ⇒ Smaller values: finer features detected



Thresholding



- Choose a threshold value t
- Set any pixels less than t to zero (off)
- Set any pixels greater than or equal to t to one (on)





Original image





Gradient magnitude image





Lower threshold





Higher threshold



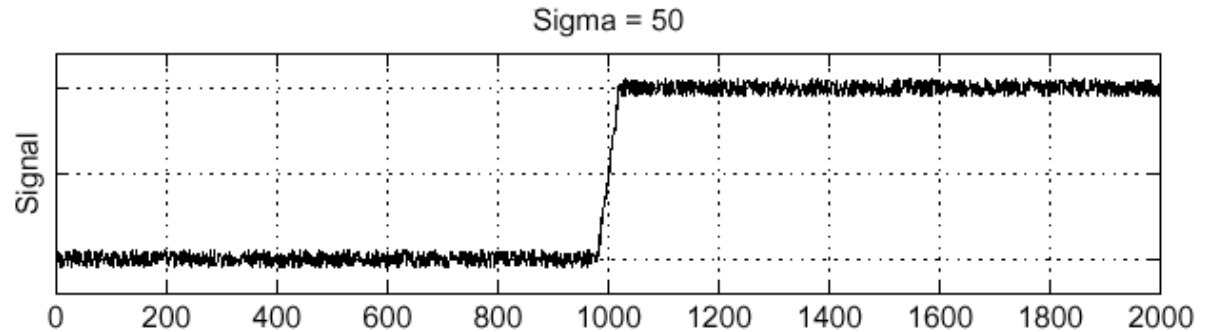


Laplacian of Gaussian

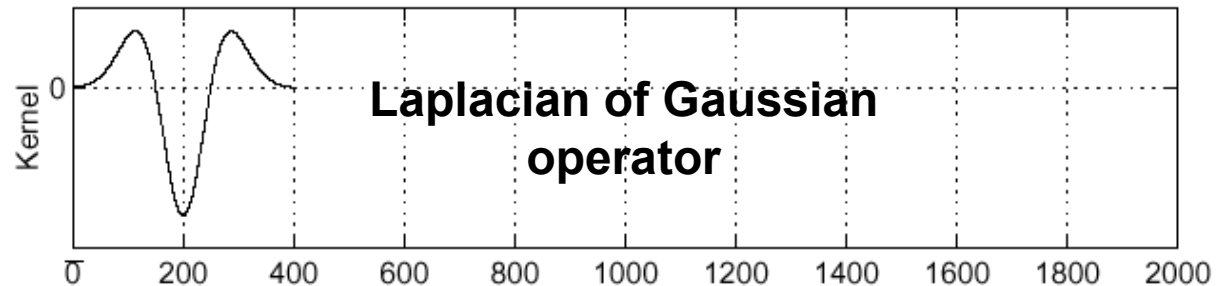


Consider $\frac{\partial^2}{\partial x^2}(h \star f)$

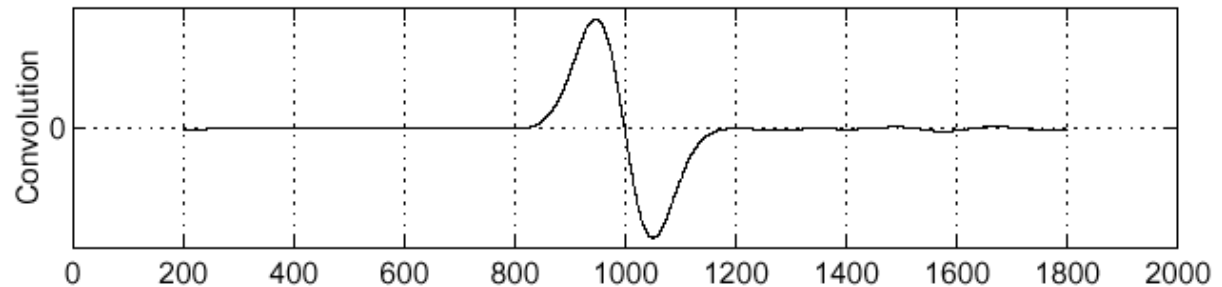
f



$\frac{\partial^2}{\partial x^2}h$



$(\frac{\partial^2}{\partial x^2}h) \star f$

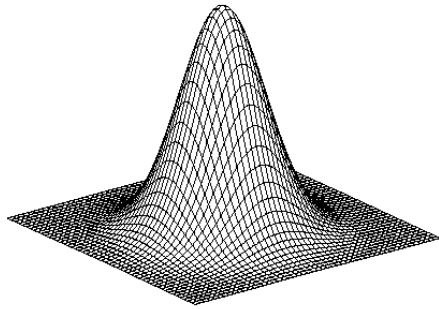


Where is the edge?

Zero-crossings of bottom graph

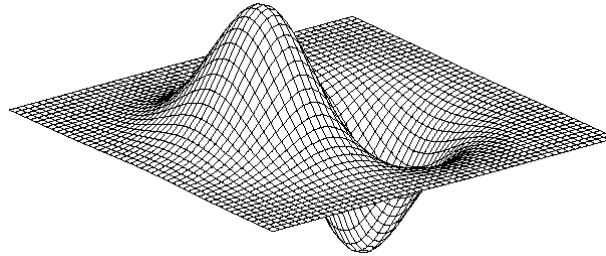


2D edge detection filters



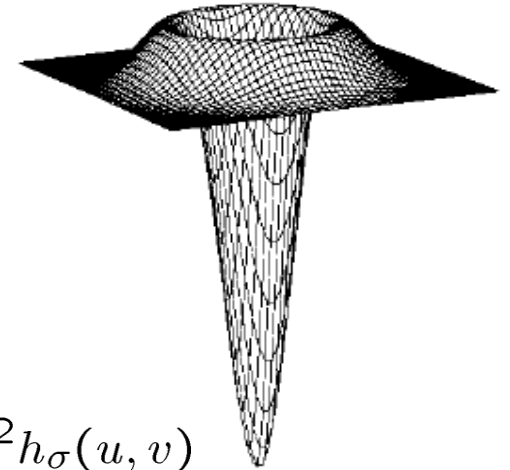
Gaussian

$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



derivative of Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$



$$\nabla^2 h_{\sigma}(u, v)$$

Laplacian of Gaussian

- ∇^2 is the Laplacian operator:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Slide credit: Steve Seitz



Today's class



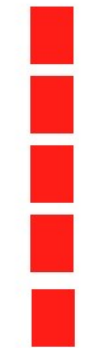
- Introduction to edge detection
- Gradients and edges
- **Canny edge detector**
- Object contour
- Pb edge detector
- Straight line detection



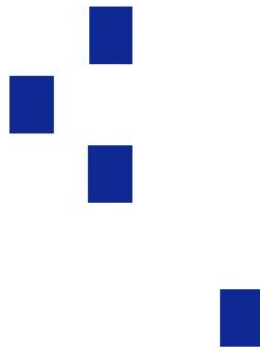
Designing an edge detector



- Criteria for an “optimal” edge detector:
 - **Good detection:** minimize the probability of false positives (detecting spurious edges caused by noise), as well as that of false negatives (missing real edges)
 - **Good localization:** must be as close as possible to the true edges
 - **Single response:** the detector must return one point only for each true edge point;



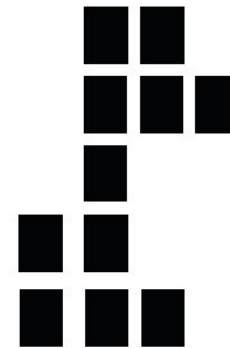
True
edge



Poor robustness
to noise



Poor
localization



Too many
responses

Basic Comparisons of Edge Operators

Gradient:

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

Good Localization
Noise Sensitive
Poor Detection

Roberts (2 x 2):

0	1
-1	0

1	0
0	-1

Sobel (3 x 3):

-1	0	1
-1	0	1
-1	0	1

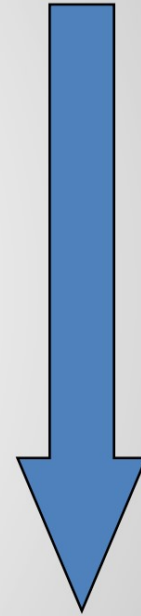
1	1	1
0	0	0
-1	-1	1

Sobel (5 x 5):

-1	-2	0	2	1
-2	-3	0	3	2
-3	-5	0	5	3
-2	-3	0	3	2
-1	-2	0	2	1

1	2	3	2	1
2	3	5	3	2
0	0	0	0	0
-2	-3	-5	-3	-2
-1	-2	-3	-2	-1

Poor Localization
Less Noise Sensitive
Good Detection





Canny edge detector



- This is probably the most widely used edge detector in computer vision
- Theoretical model: step-edges corrupted by additive Gaussian noise
- Canny has shown that the first derivative of the Gaussian closely approximates the operator that optimizes the product of *signal-to-noise ratio* and localization

J. Canny, *[A Computational Approach To Edge Detection](#)*, IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986.



Canny edge detector



- Filter image with derivative of Gaussian
- Find magnitude and orientation of gradient
- **Non-maximum suppression:**
 - Thin wide “ridges” down to single pixel width
- **Linking and thresholding (hysteresis):**
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them
- MATLAB: `edge(image, 'canny');`
- `>>help edge`



The Canny edge detector



original image (Lena)



The Canny edge detector



norm of the gradient



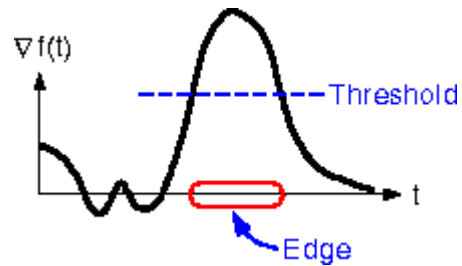
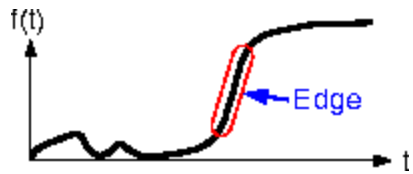
The Canny edge detector



thresholding



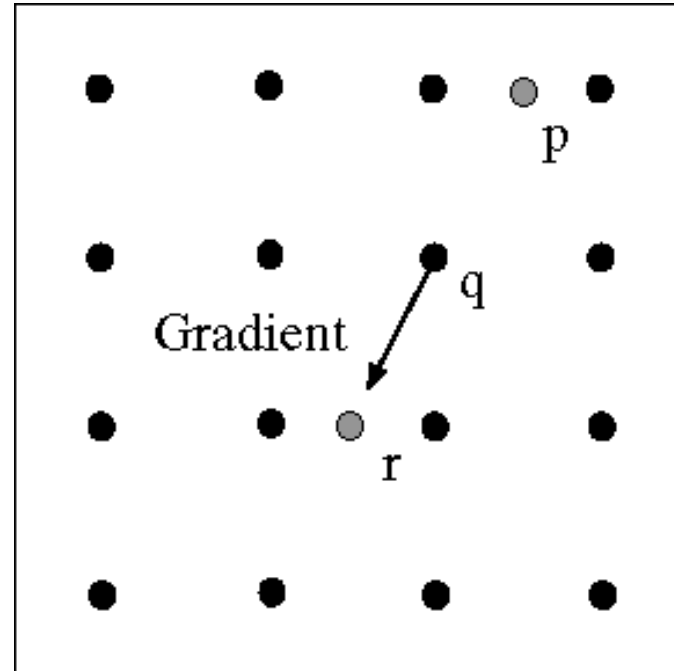
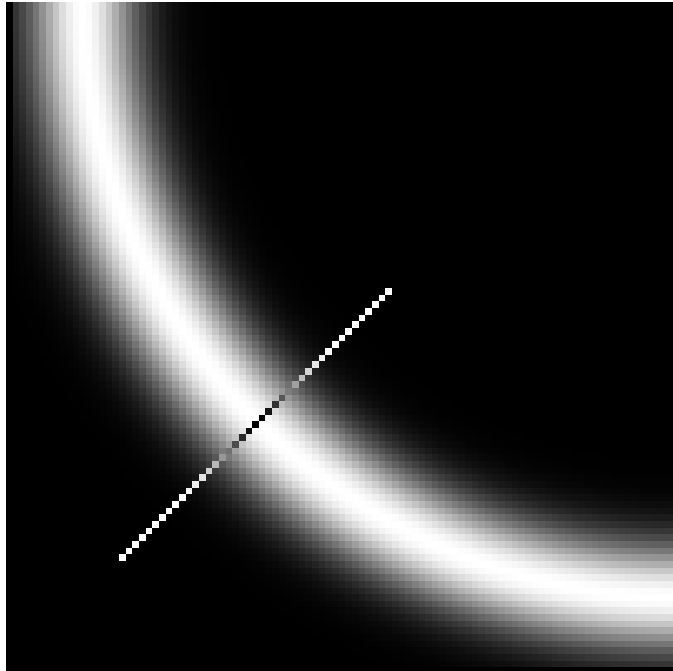
The Canny edge detector



How to turn these thick regions of the gradient into curves?



Non-maximum suppression



Check if pixel is local maximum along gradient direction,
select single max across width of the edge

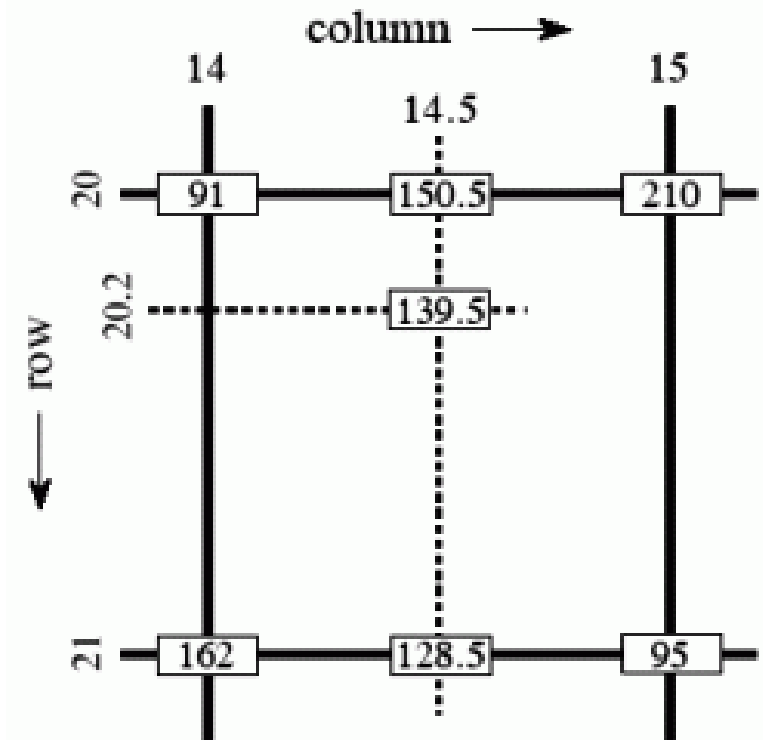
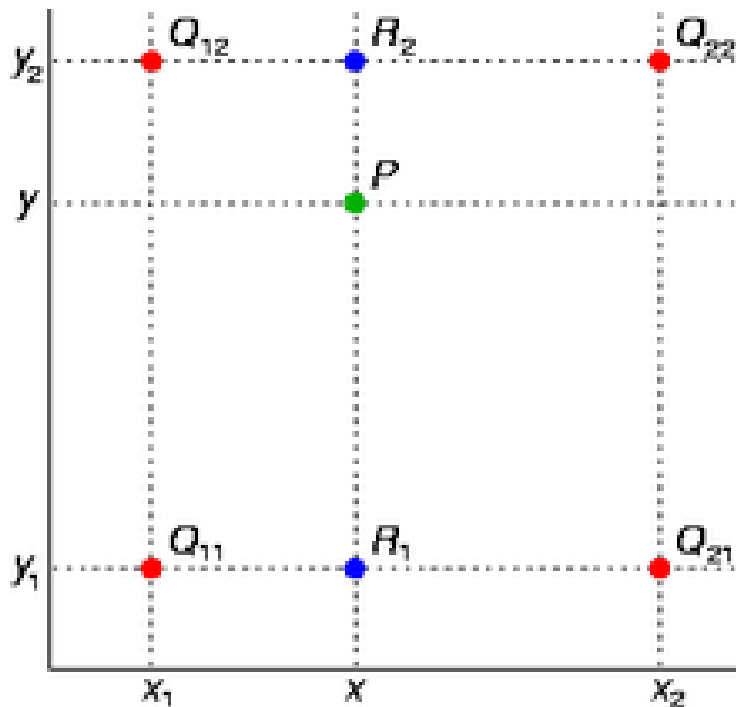
- requires checking interpolated pixels p and r



Bilinear Interpolation

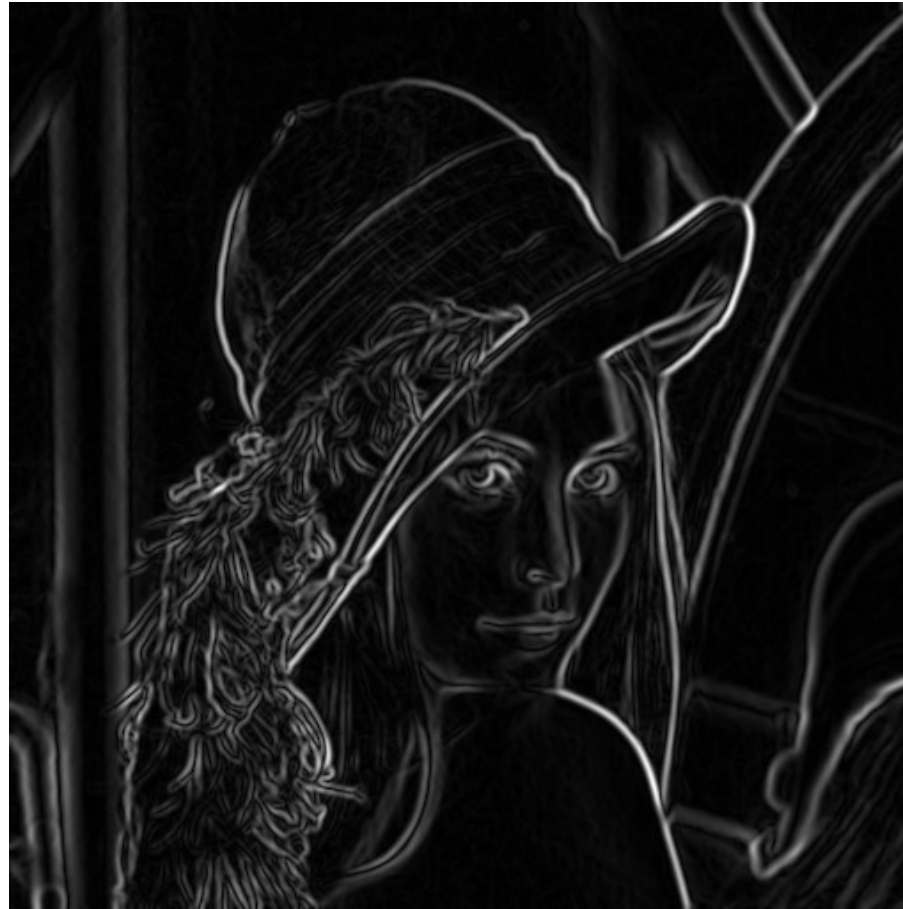


$$f(x, y) \approx \begin{bmatrix} 1 - x & x \end{bmatrix} \begin{bmatrix} f(0, 0) & f(0, 1) \\ f(1, 0) & f(1, 1) \end{bmatrix} \begin{bmatrix} 1 - y \\ y \end{bmatrix}.$$



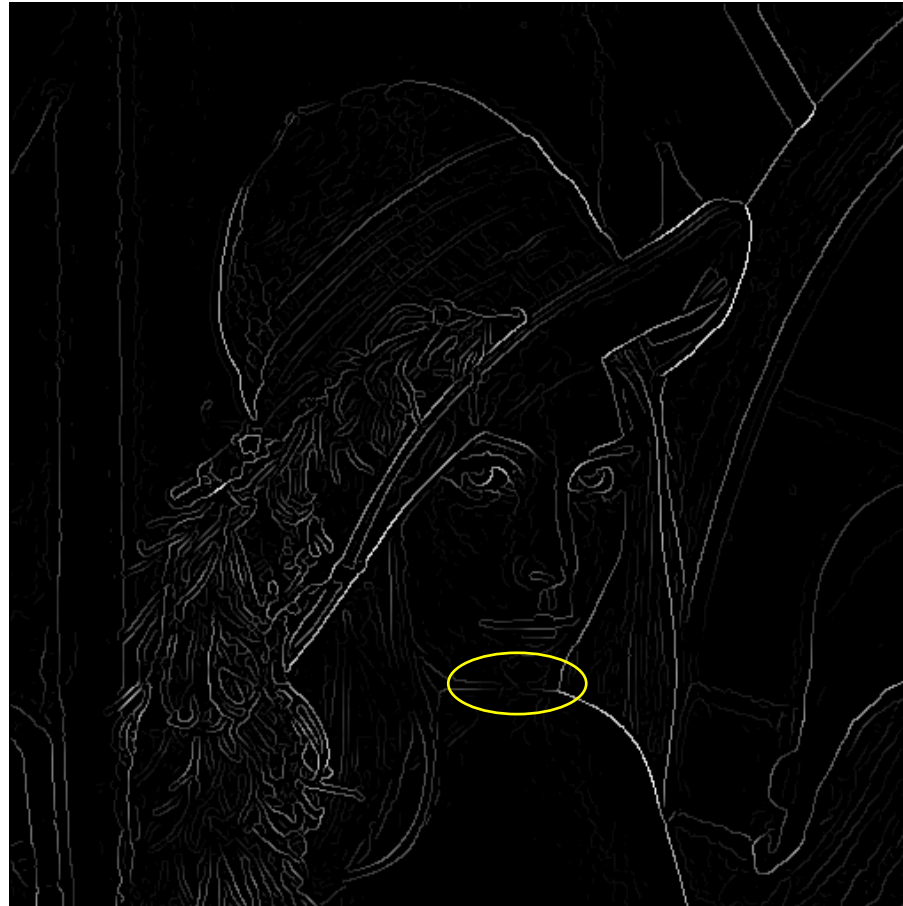


Before non-max suppression





After non-max suppression



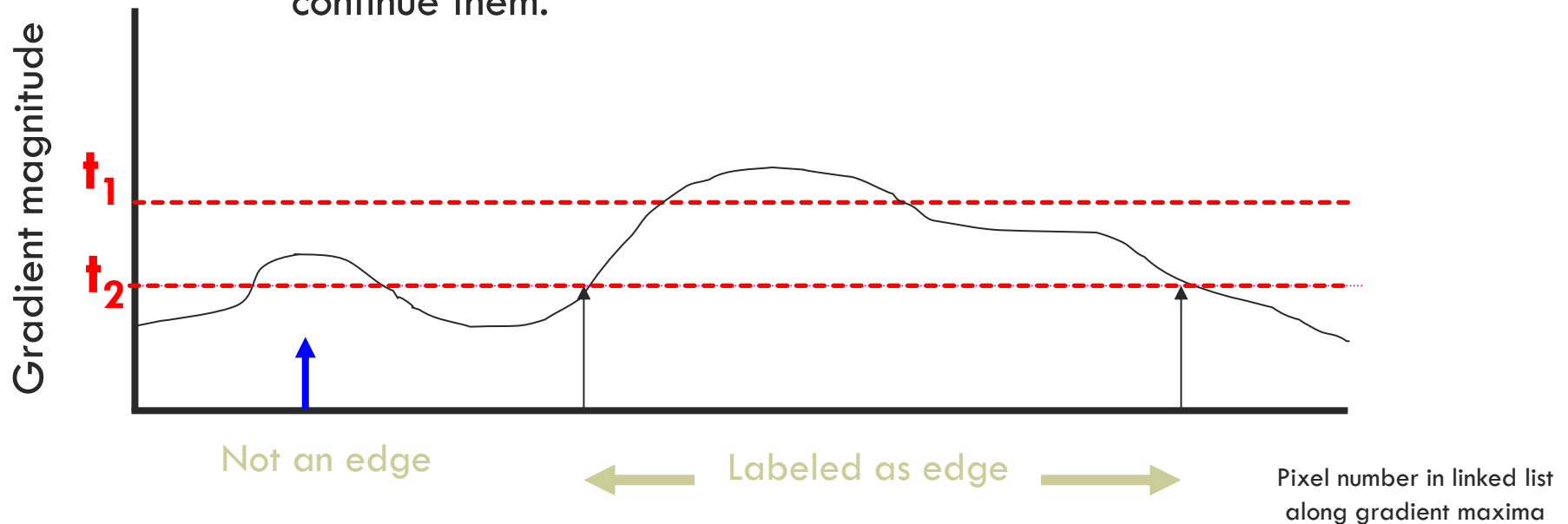
Problem:
pixels along
this edge
didn't
survive the
thresholding



Closing edge gaps

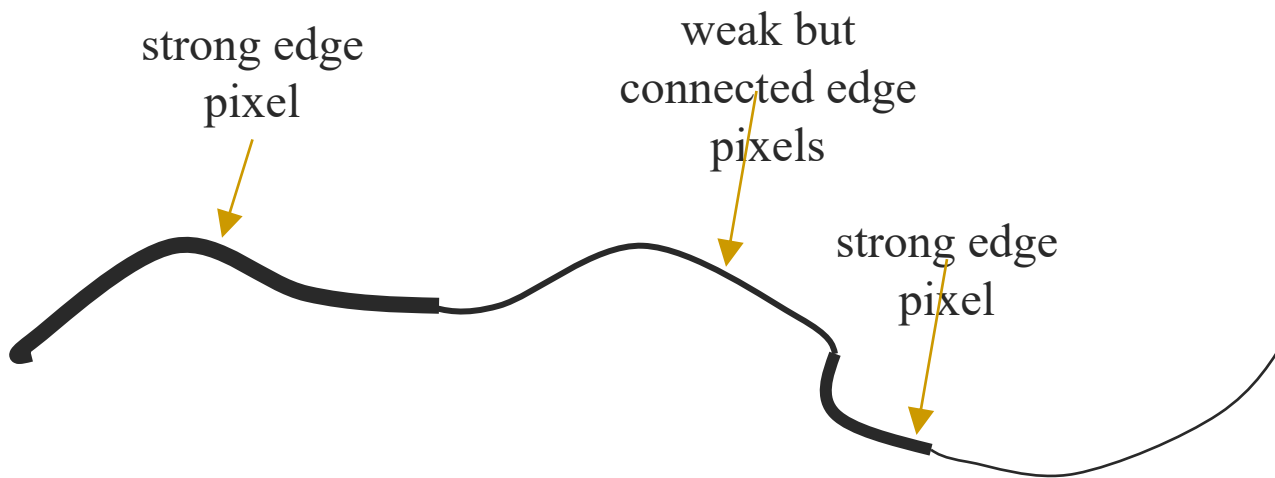


- Check that maximum value of gradient value is sufficiently large
 - use **hysteresis**
 - use a high threshold to start edge curves and a low threshold to continue them.





Hysteresis thresholding



Source: S. Seitz



Hysteresis thresholding



- Threshold at low/high levels to get weak/strong edge pixels
- Do connected components, starting from strong edge pixels





Final Canny Edges





Effect of σ (Gaussian kernel spread/size)



original



Canny with $\sigma = 1$



Canny with $\sigma = 2$

The choice of σ depends on desired behavior

- large σ detects large scale edges
- small σ detects fine features



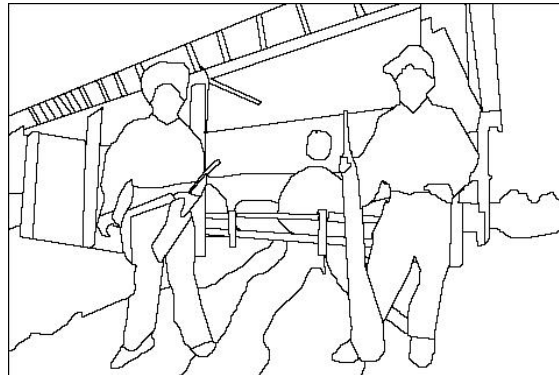
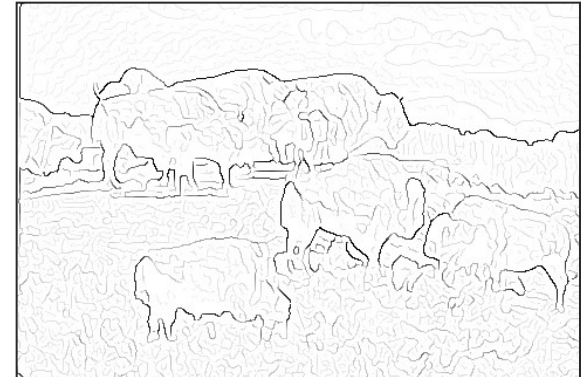
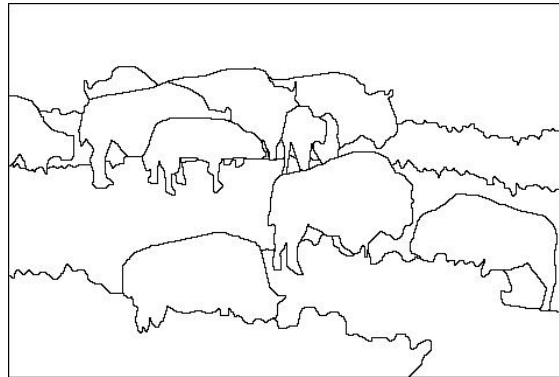
Low-level edges vs. perceived contours



image

human segmentation

gradient magnitude



- **Berkeley segmentation database:**
<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench/>



A Database of Human Segmented Natural Images and its Application to Evaluating Segmentation Algorithms and Measuring Ecological Statistics



David Martin Charles Fowlkes Doron Tal Jitendra Malik
Department of Electrical Engineering and Computer Sciences
University of California, Berkeley
Berkeley, CA 94720
{dmartin,fowlkes,doron,malik}@eecs.berkeley.edu

This paper presents a database containing 'ground truth' segmentations produced by humans for images of a wide variety of natural scenes. We define an error measure which quantifies the consistency between segmentations of differing granularities and find that different human segmentations of the same image are highly consistent. Use of this dataset is demonstrated in two applications: (1) evaluating the performance of segmentation algorithms and (2) measuring probability distributions associated with Gestalt grouping factors as well as statistics of image region properties.

A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics

Proceedings Eighth IEEE International Conference on ..., 2001 - ieeexplore.ieee.org

This paper presents a database containing 'ground truth' segmentations produced by humans for images of a wide variety of natural scenes. We define an error measure which quantifies ...

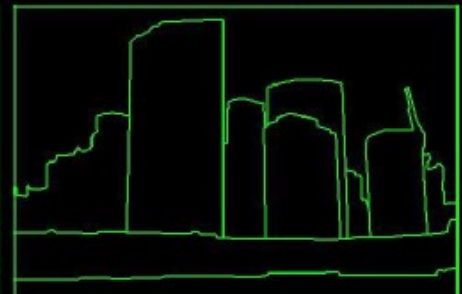
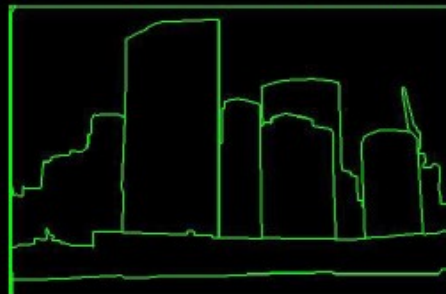
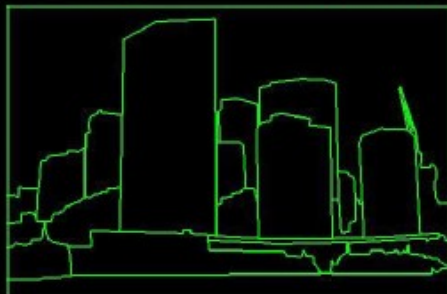
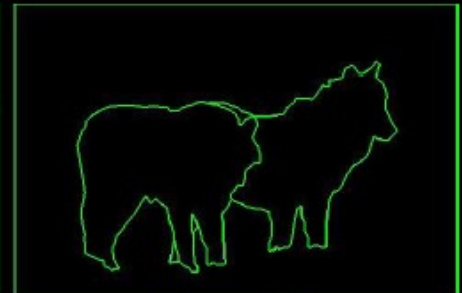
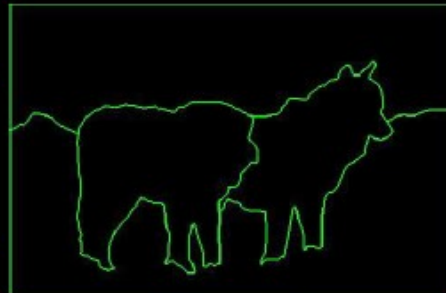
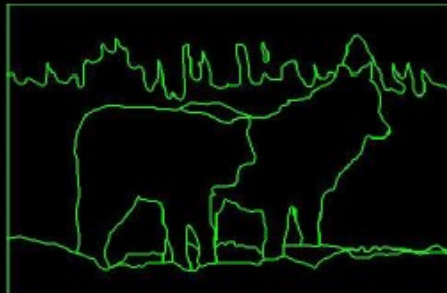
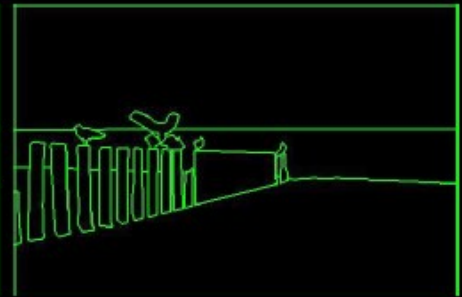
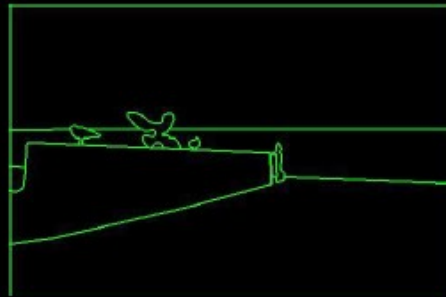
☆ Notes 🔍 References Cited by 9120 Related articles



Protocol

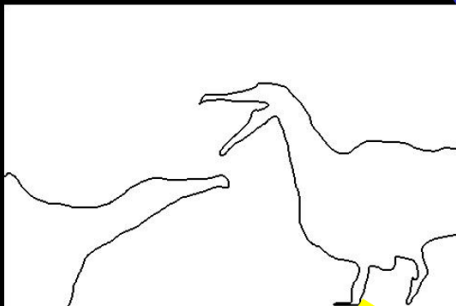
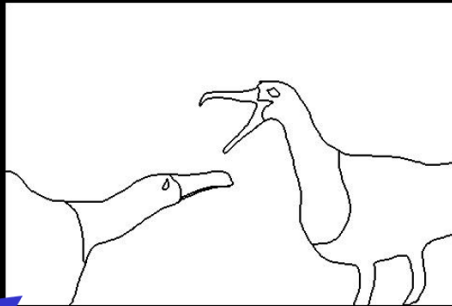
You will be presented a photographic image. Divide the image into some number of segments, where the segments represent “things” or “parts of things” in the scene. The number of segments is up to you, as it depends on the image. Something between 2 and 30 is likely to be appropriate. It is important that all of the segments have approximately equal importance.

- Custom segmentation tool
- Subjects obtained from work-study program (UC Berkeley undergraduates)

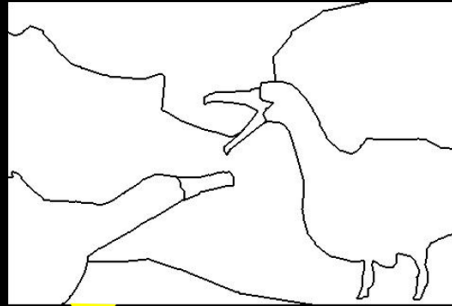


Consistency

A



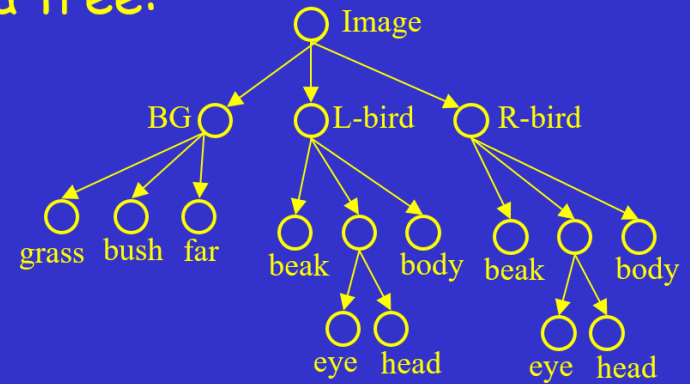
B



C

- A,C are refinements of B
- A,C are mutual refinements
- A,B,C represent the same percept
 - Attention accounts for differences

Perceptual organization forms a tree:



- ★ Two segmentations are consistent when they can be explained by the same segmentation tree (i.e. they could be derived from a single perceptual organization).

Dataset Summary

- 30 subjects, age 19-23
 - 17 men, 13 women
 - 9 with artistic training
- 8 months
- 1,458 person hours
- 1,020 Corel images
- 11,595 Segmentations
 - 5,555 color, 5,554 gray, 486 inverted/negated



Today's class



- Introduction to edge detection
- Gradients and edges
- Canny edge detector
- Object contour
- **Pb edge detector**
- Straight line detection



Learning to Detect Natural Image Boundaries Using Local Brightness, Color, and Texture Cues

David R. Martin, *Member, IEEE*, Charless C. Fowlkes, and Jitendra Malik, *Member, IEEE*

Abstract—The goal of this work is to accurately detect and localize boundaries in natural scenes using local image measurements. We formulate features that respond to characteristic changes in brightness, color, and texture associated with natural boundaries. In order to combine the information from these features in an optimal way, we train a classifier using human labeled images as ground truth. The output of this classifier provides the posterior probability of a boundary at each image location and orientation. We present precision-recall curves showing that the resulting detector significantly outperforms existing approaches. Our two main results are 1) that cue combination can be performed adequately with a simple linear model and 2) that a proper, explicit treatment of texture is required to detect boundaries in natural images.

Index Terms—Texture, supervised learning, cue combination, natural images, ground truth segmentation data set, boundary detection, boundary localization.



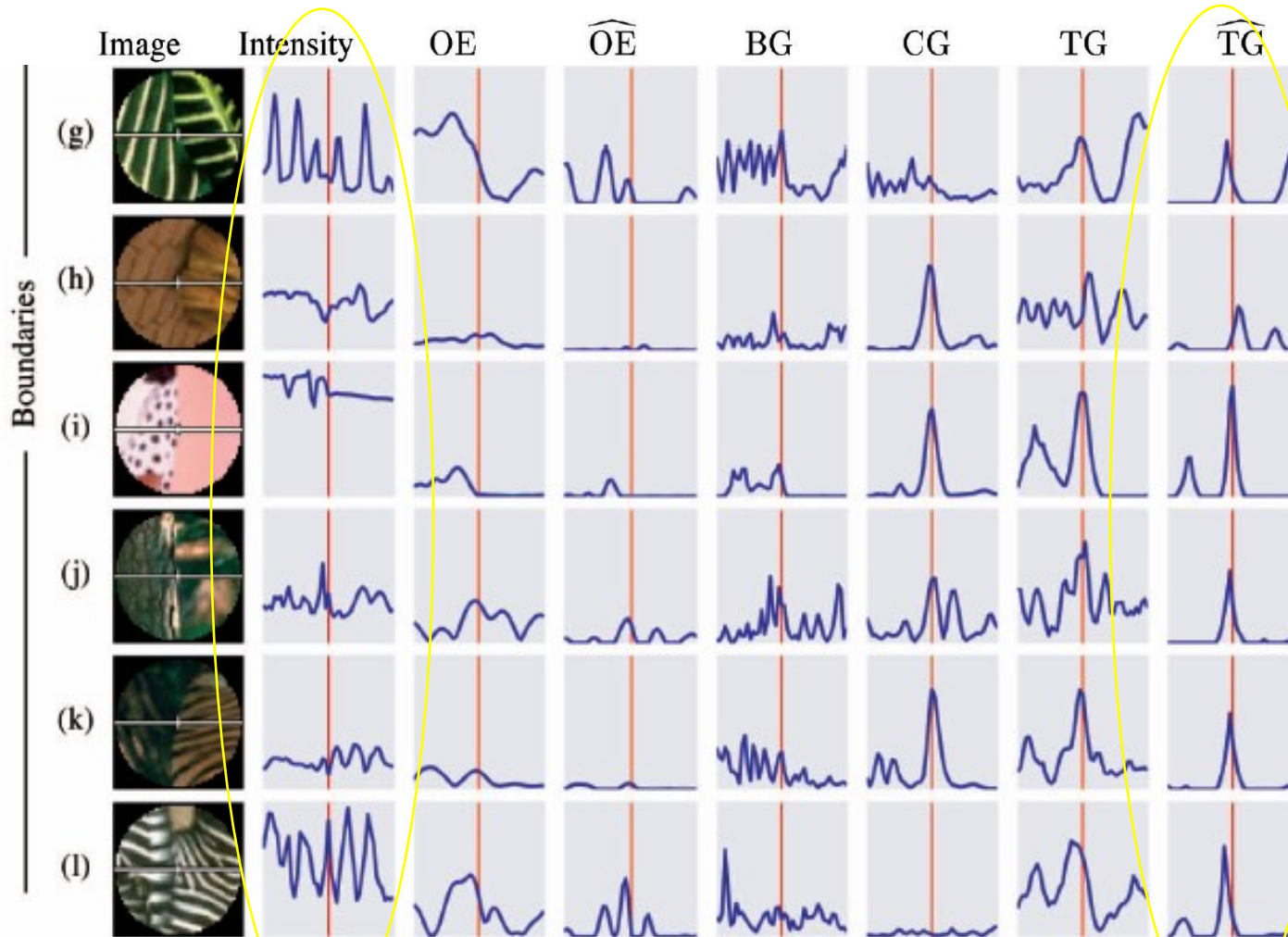


Learn from
humans **which**
combination of
features is most
indicative of a
“good” contour?





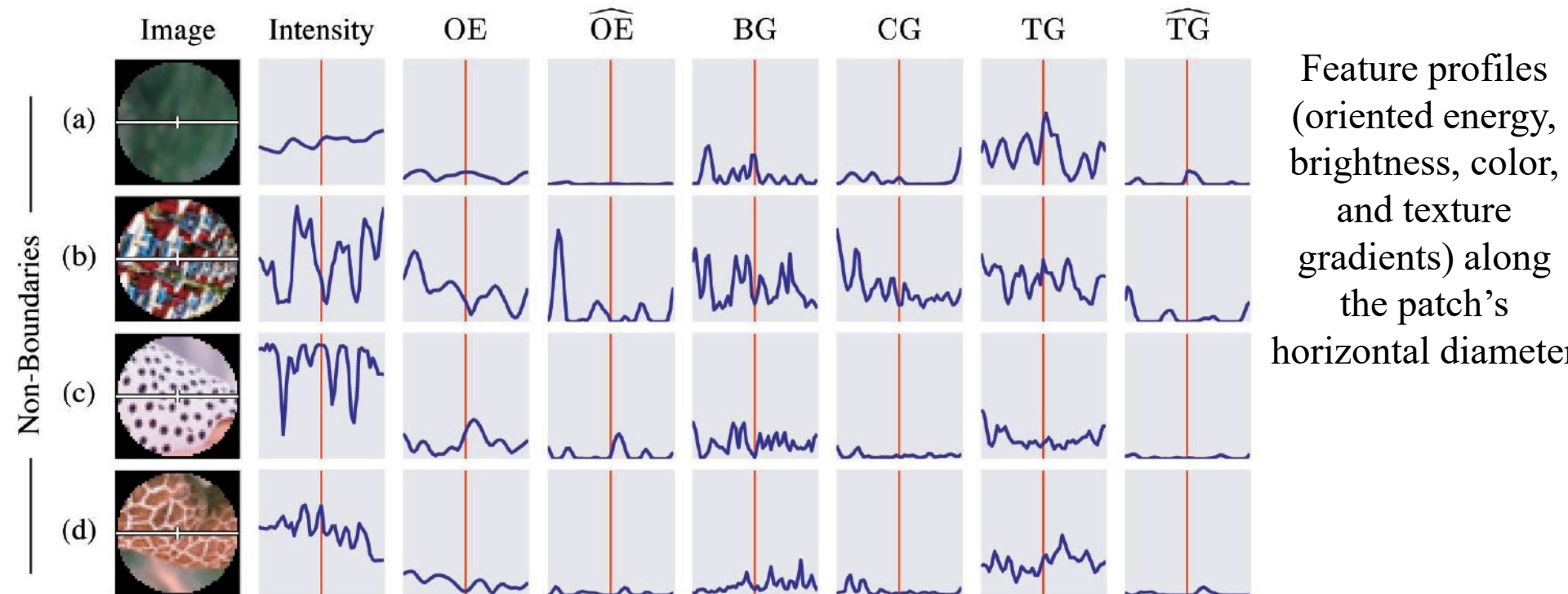
What features are responsible for perceived edges?



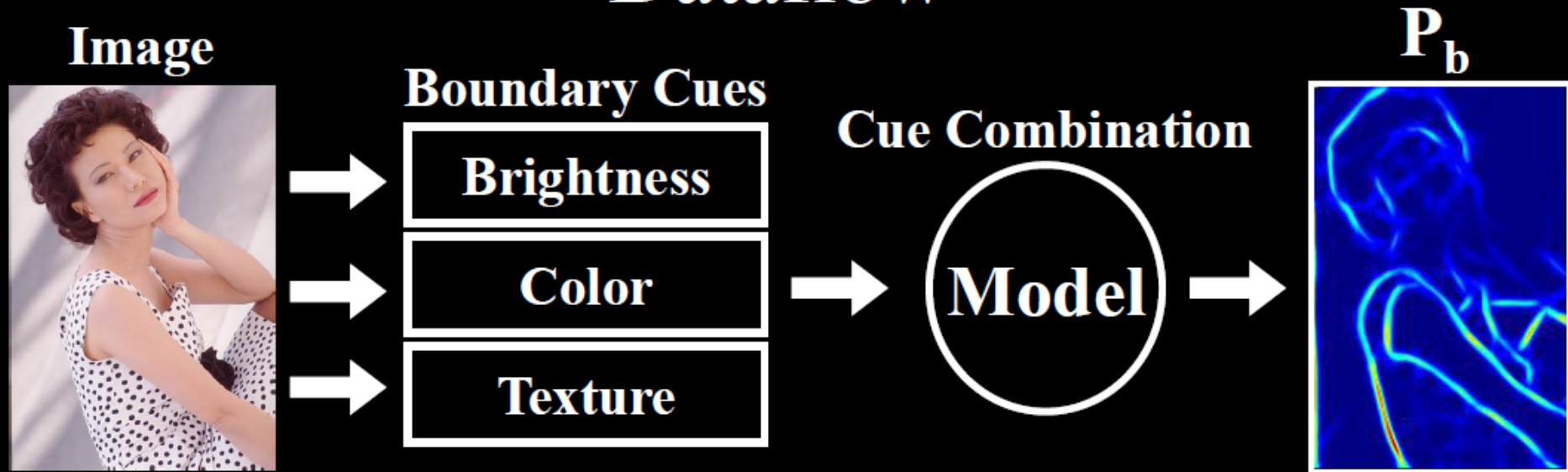
Feature profiles (oriented energy, brightness, color, and texture gradients) along the patch's horizontal diameter



What features are responsible for perceived edges?



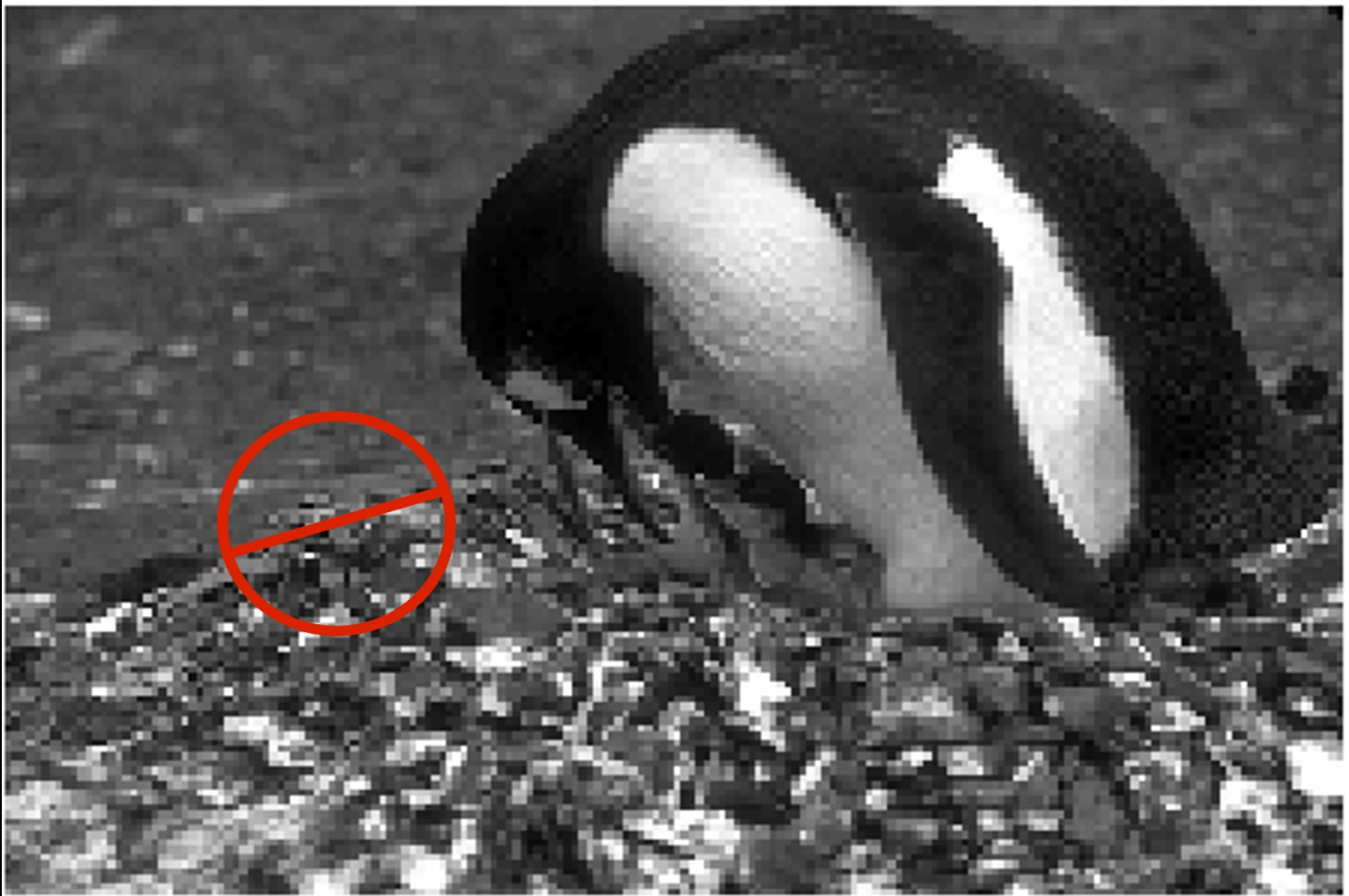
Dataflow



Challenges: texture cue, cue combination

Goal: learn the posterior probability of a boundary $P_b(x,y,\theta)$ from local information only

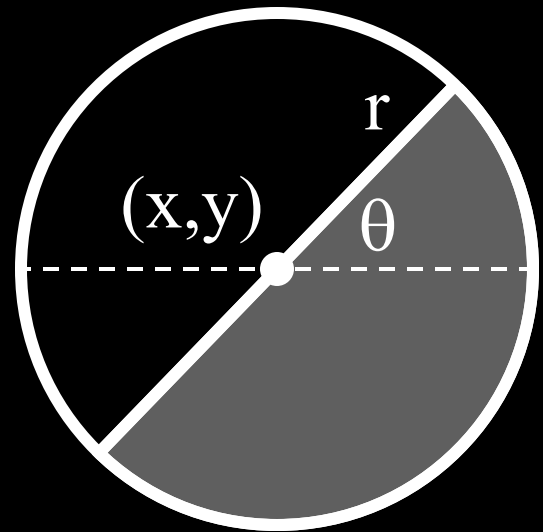
Oriented Feature Gradient



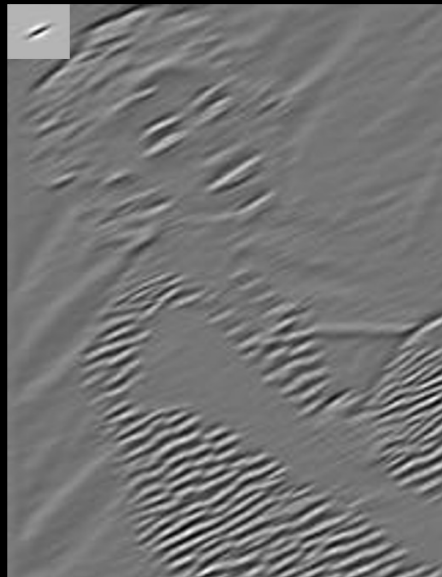
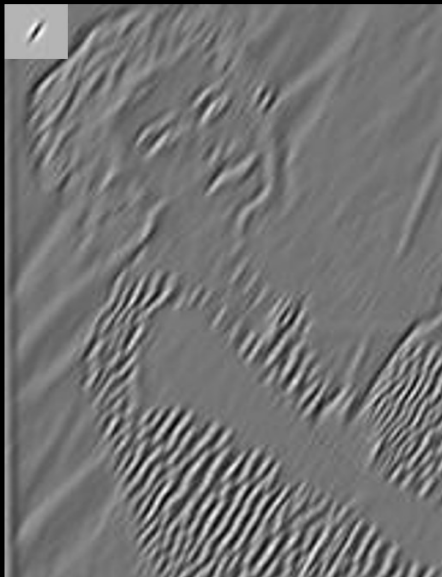
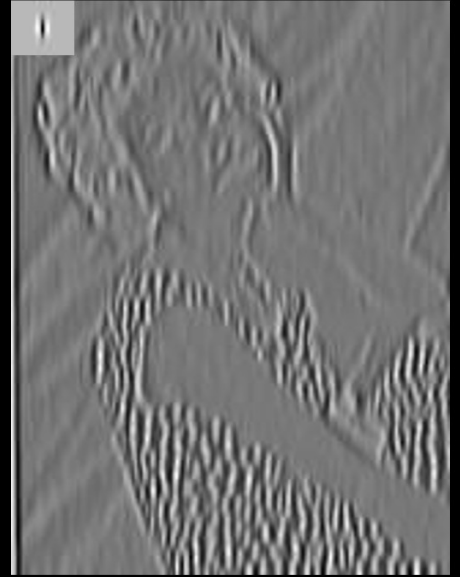
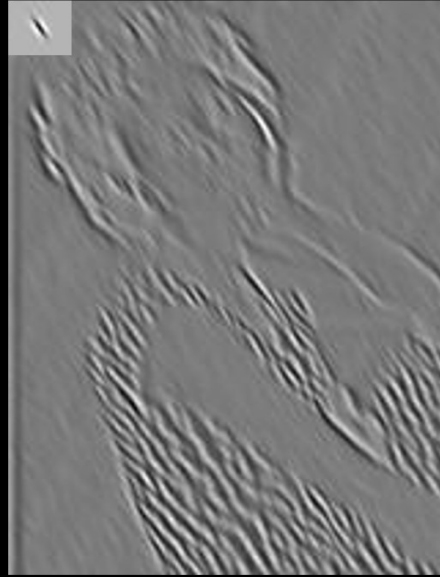
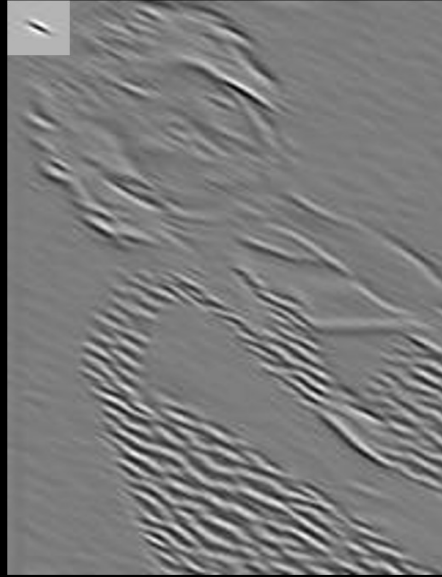
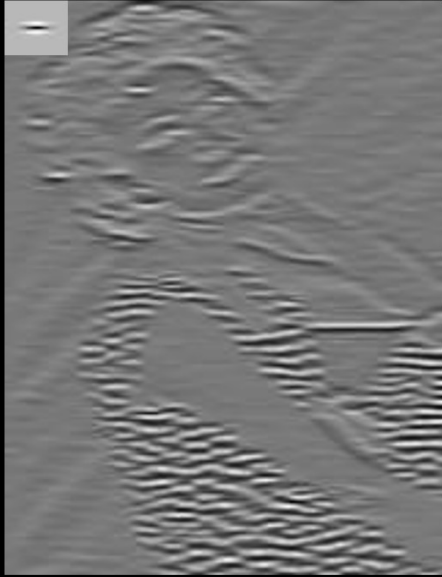
Brightness and Color Features

- 1976 CIE L*a*b* colorspace
- Brightness Gradient BG(x,y,r,θ)
 - χ^2 difference in L* distribution
- Color Gradient CG(x,y,r,θ)
 - χ^2 difference in a* and b* distributions

$$\chi^2(g, h) = \frac{1}{2} \sum_i \frac{(g_i - h_i)^2}{g_i + h_i}$$



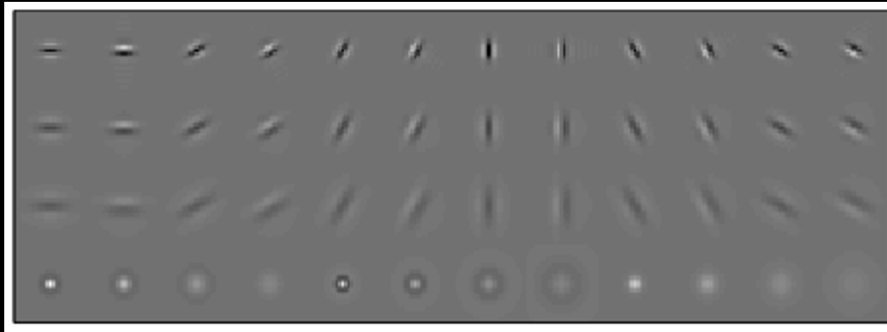
Filter Outputs



2D Textons

- Goal: find canonical local features in a texture;

1) Filter image with linear filters:

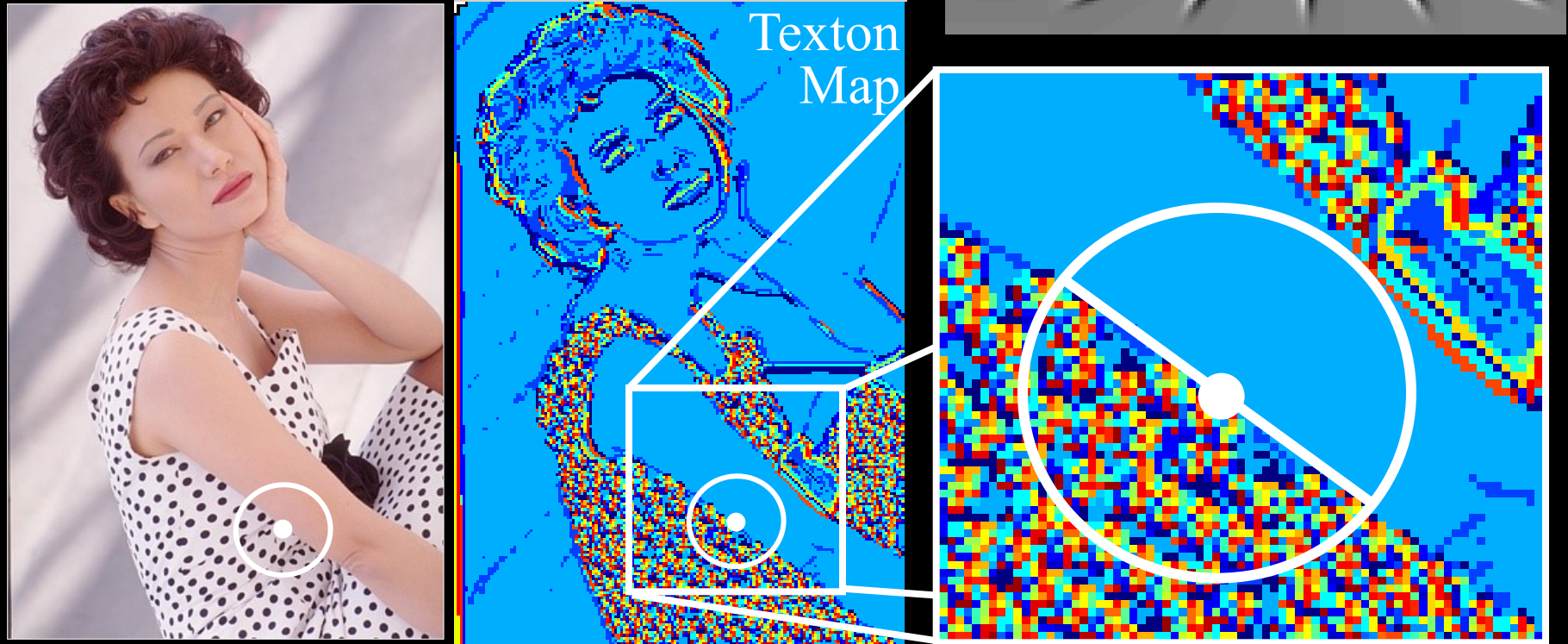


2) Vector quantization (k-means) on filter outputs;

3) Quantization centers are the textons.

- Spatial distribution of textons defines the texture;

Texture Feature



- Texture Gradient TG (x, y, r, θ)
 - χ^2 difference of texton histograms
 - Textons are vector-quantized filter outputs

Cue Combination Models

- Classification Trees
 - Top-down splits to maximize entropy, error bounded
 - Density Estimation
 - Adaptive bins using k-means
 - Logistic Regression, 3 variants
 - Linear and quadratic terms
 - Confidence-rated generalization of AdaBoost (Schapire&Singer)
 - Hierarchical Mixtures of Experts (Jordan&Jacobs)
 - Up to 8 experts, initialized top-down, fit with EM
 - Support Vector Machines (`libsvm`, Chang&Lin)
 - Gaussian kernel, ν -parameterization
- Range over bias, complexity, parametric/non-parametric

Dataflow

Image



Optimized Cues

Brightness

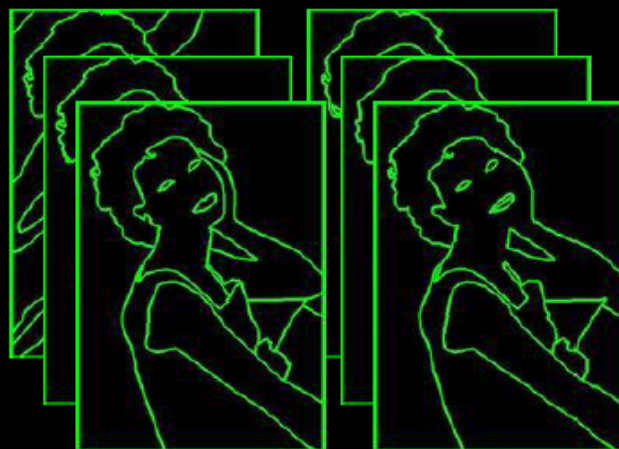
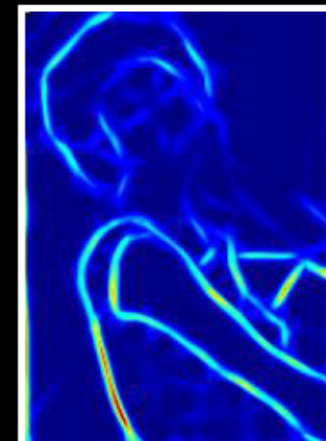
Color

Texture

Cue Combination

Model

P_b



Human Segmentations

Benchmark

Computing Precision/Recall

Recall = $\Pr(\text{signal}|\text{truth})$ = fraction of ground truth found by the signal

Precision = $\Pr(\text{truth}|\text{signal})$ = fraction of signal that is correct

- Always a trade-off between the two
- Standard measures in information retrieval (van Rijsbergen XX)
- ROC from standard signal detection: the wrong approach

Strategy

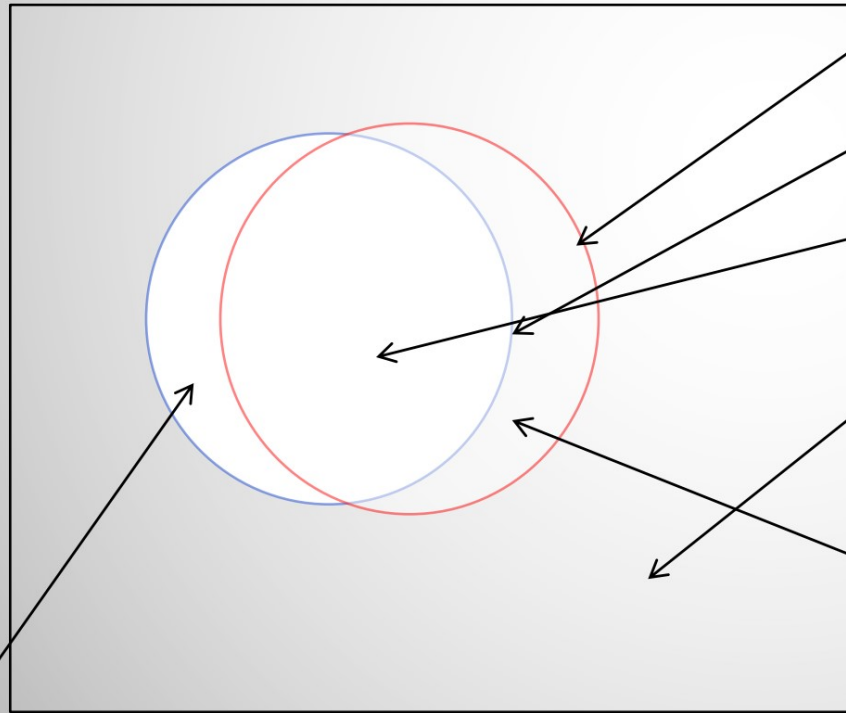
- Detector output (P_b) is a soft boundary map
- Compute precision/recall curve:
 - Threshold P_b at many points t in $[0,1]$
 - $\text{Recall} = \Pr(P_b > t | \text{seg}=1)$
 - $\text{Precision} = \Pr(\text{seg}=1 | P_b > t)$

Evaluate Edge Detection

$$\text{precision} = \frac{GT \cap RM}{RM} = \frac{TP}{RM}$$

$$\text{recall} = \frac{GT \cap RM}{GT} = \frac{TP}{GT}$$

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$



Ground Truth (GT)

Results of Method (RM)

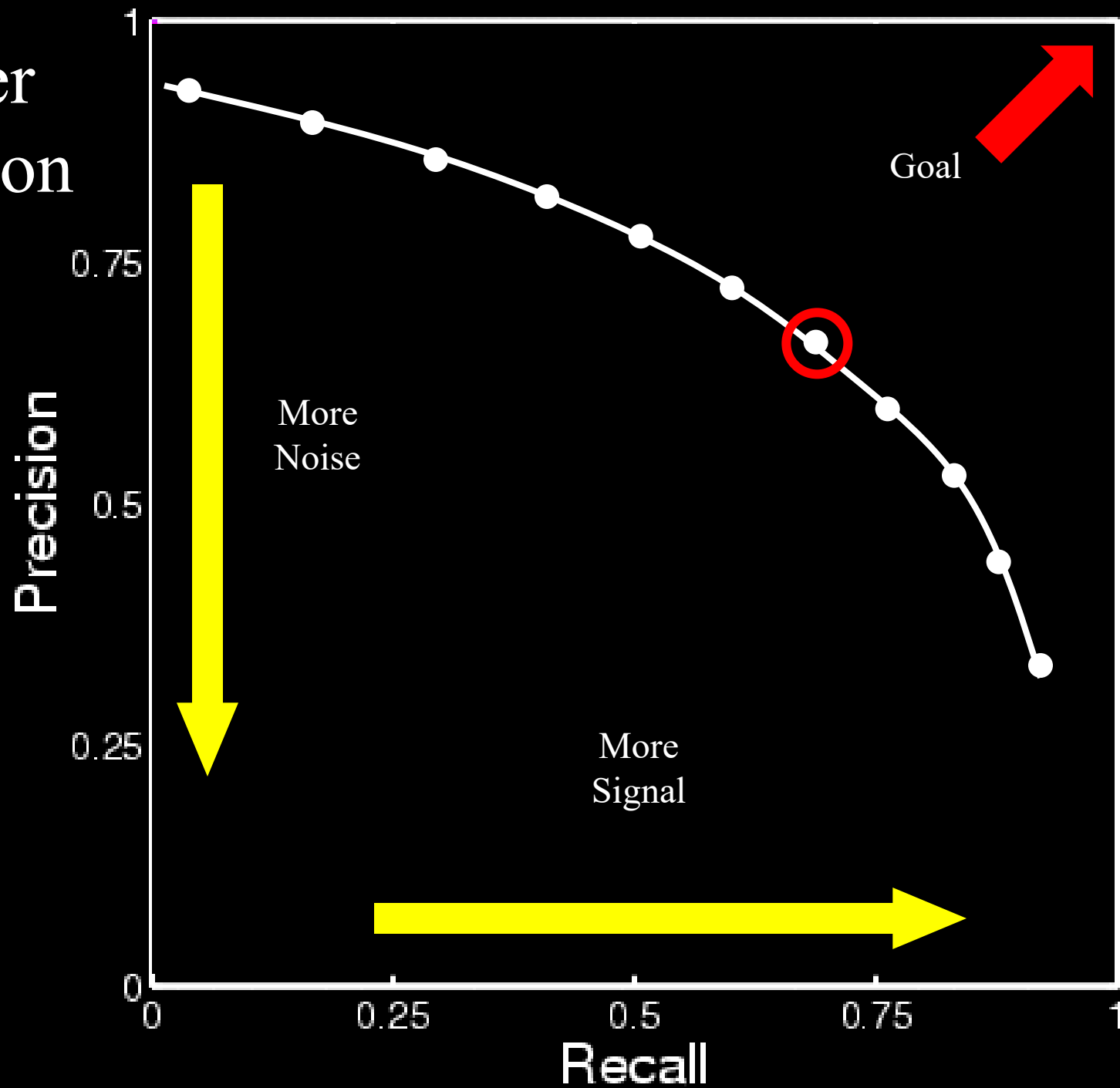
True Positives (TP)

True Negatives (TN)

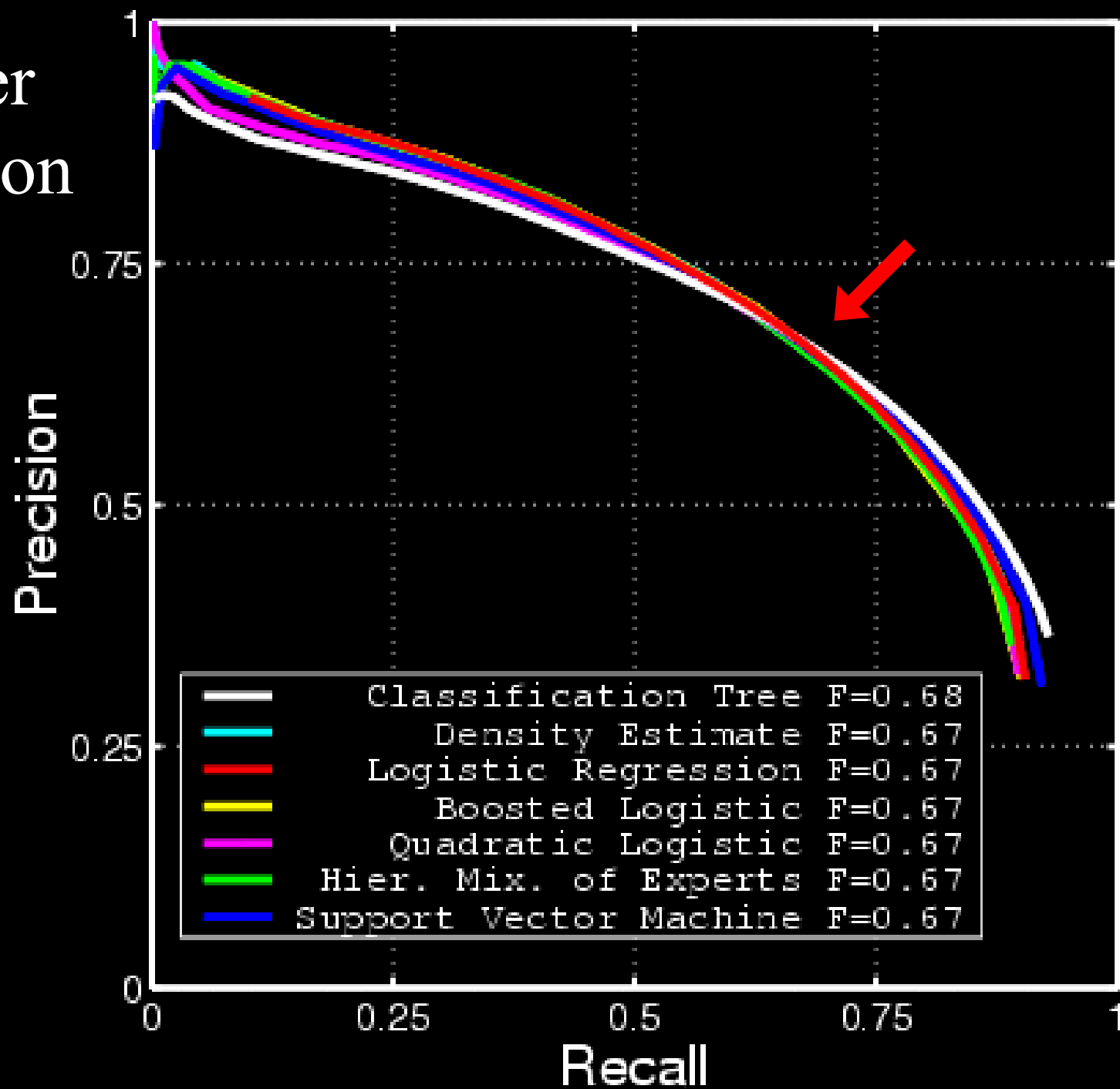
False Negatives (FN)

False Positives (FP)

Classifier Comparison

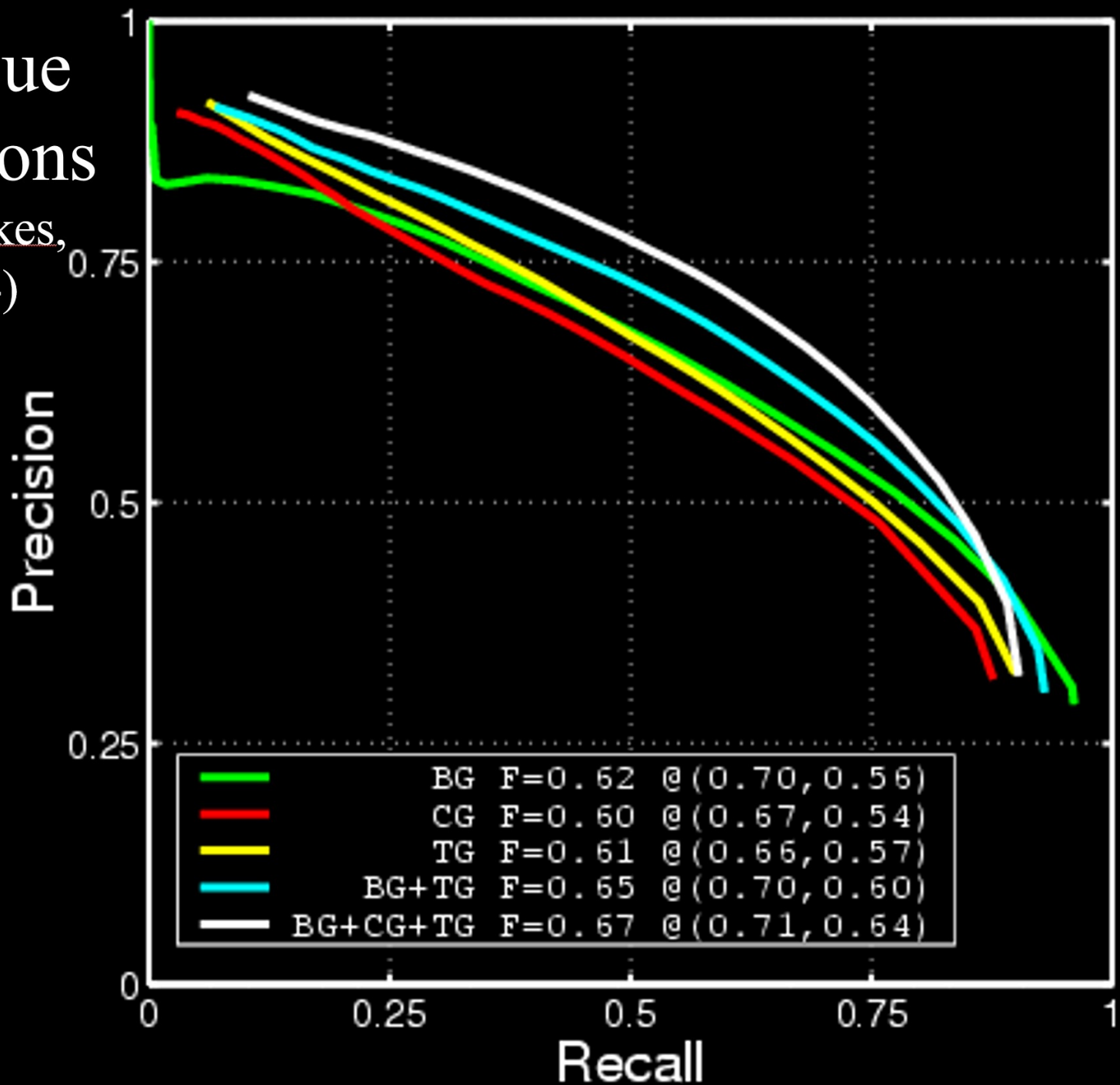


Classifier Comparison



Various Cue Combinations

(Martin, Fowlkes, Malik, 2004)



Alternate Approaches

- Canny Detector
 - Canny 1986
 - MATLAB implementation
 - With and without hysteresis
- Second Moment Matrix
 - Nitzberg/Mumford/Shiota 1993
 - cf. Förstner and Harris corner detectors
 - Used by Konishi et al. 1999 in learning framework
 - Logistic model trained on full eigenspectrum

P_b Images

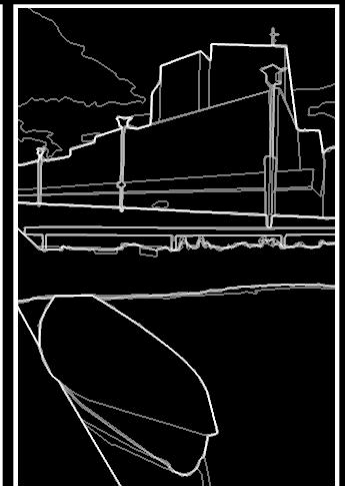
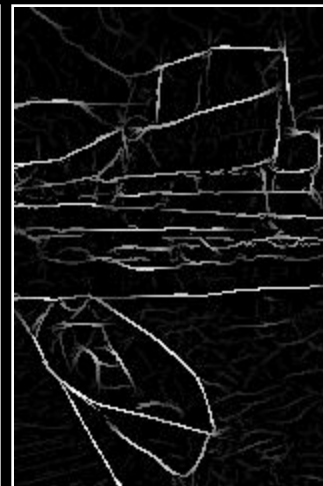
Image

Canny

2MM

Us

Human



P_b Images II

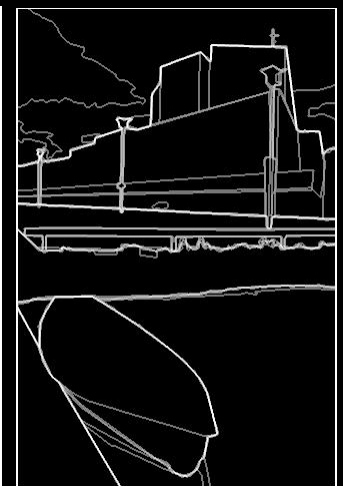
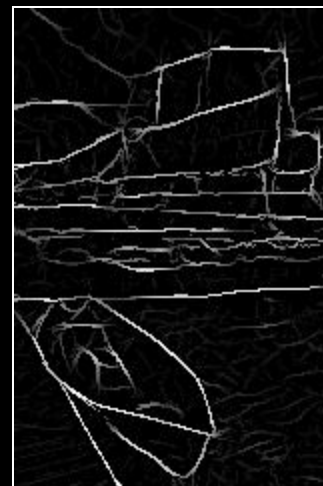
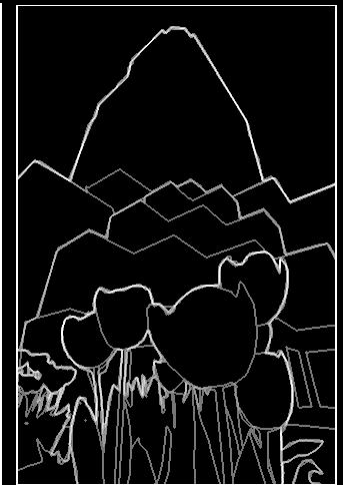
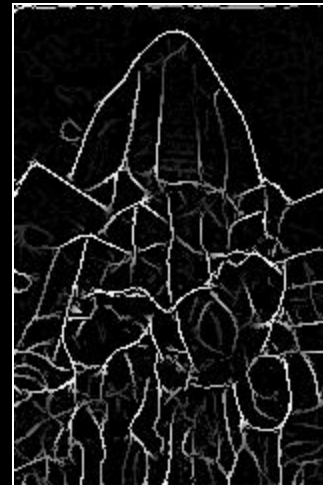
Image

Canny

2MM

Us

Human



P_b Images III

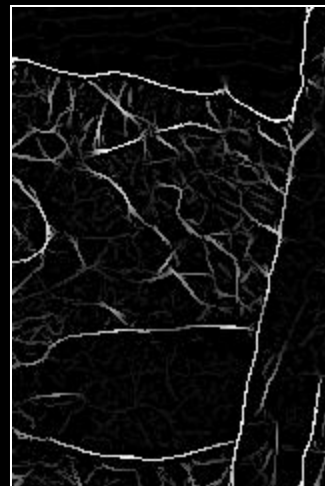
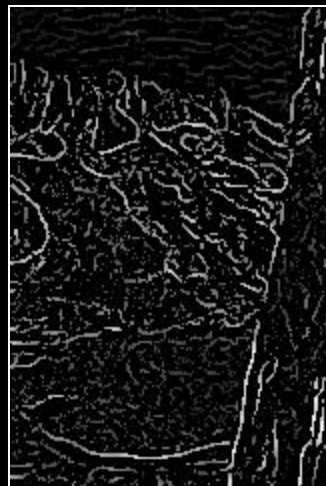
Image

Canny

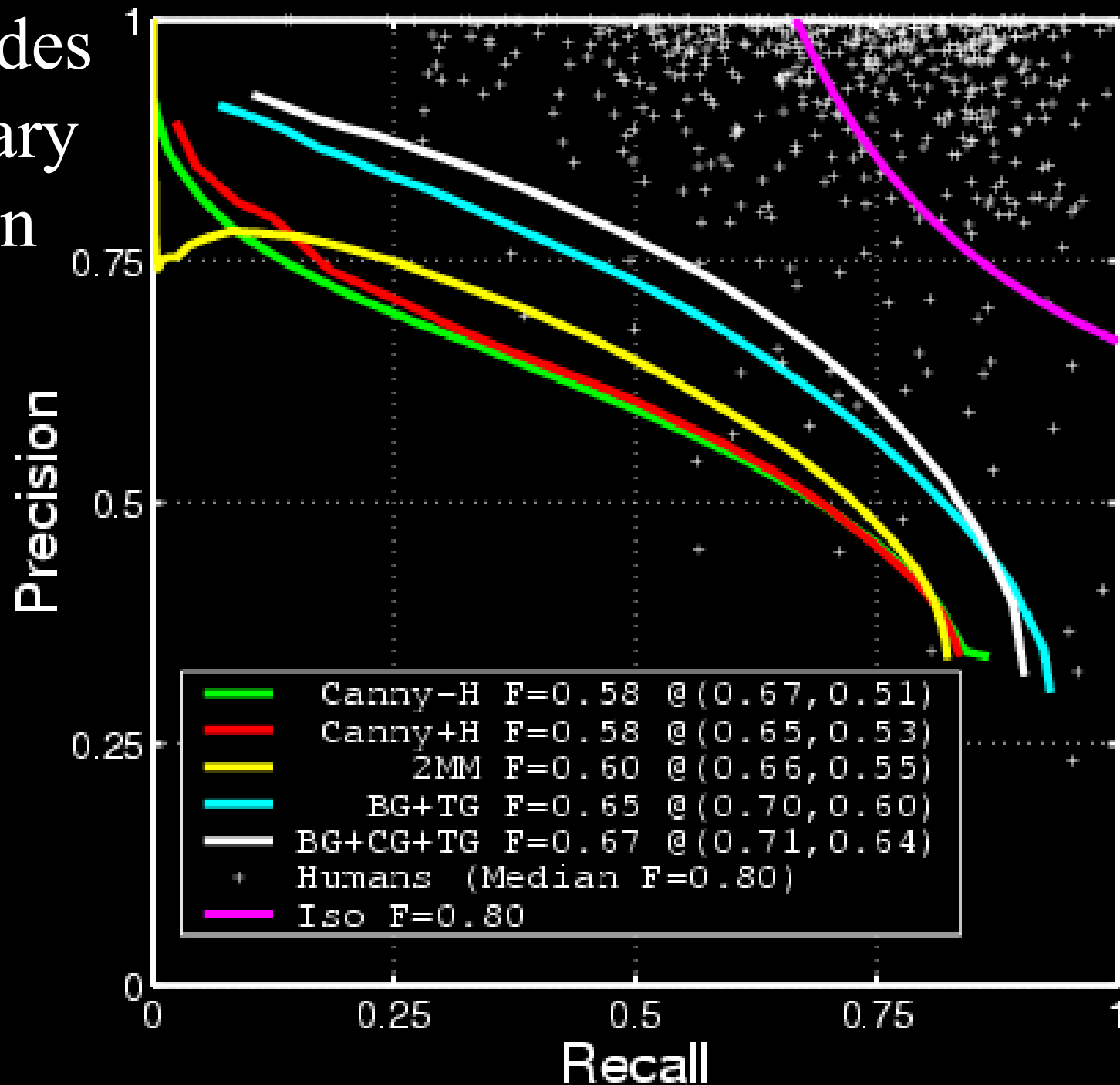
2MM

Us

Human



Two Decades of Boundary Detection



Findings

1. A simple linear model is sufficient for cue combination
 - All cues weighted approximately equally in logistic
2. Proper texture edge model is not optional for complex natural images
 - Texture suppression is not sufficient!
3. Significant improvement over state-of-the-art in boundary detection
 - $P_b(x,y,\theta)$ useful for higher-level processing
4. Empirical approach critical for both cue calibration and cue combination



Brightness

Color

Texture

Combined

Human

Image

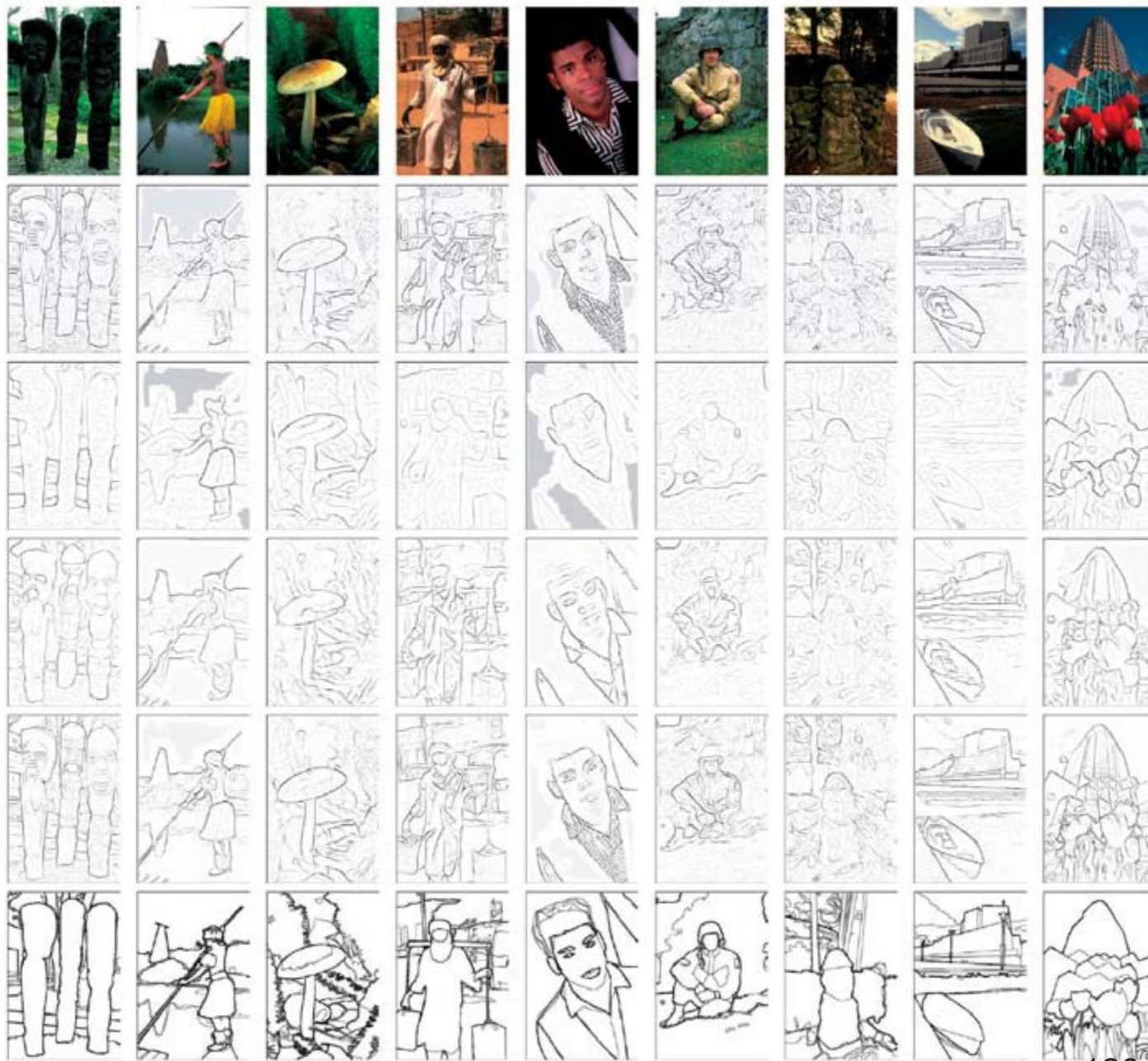
BG

CG

TG

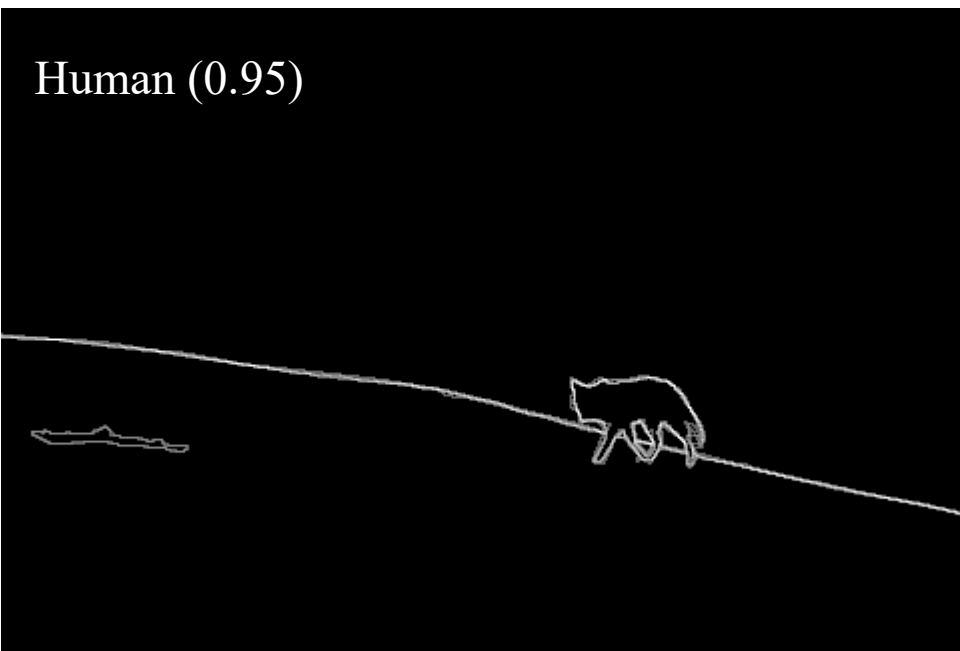
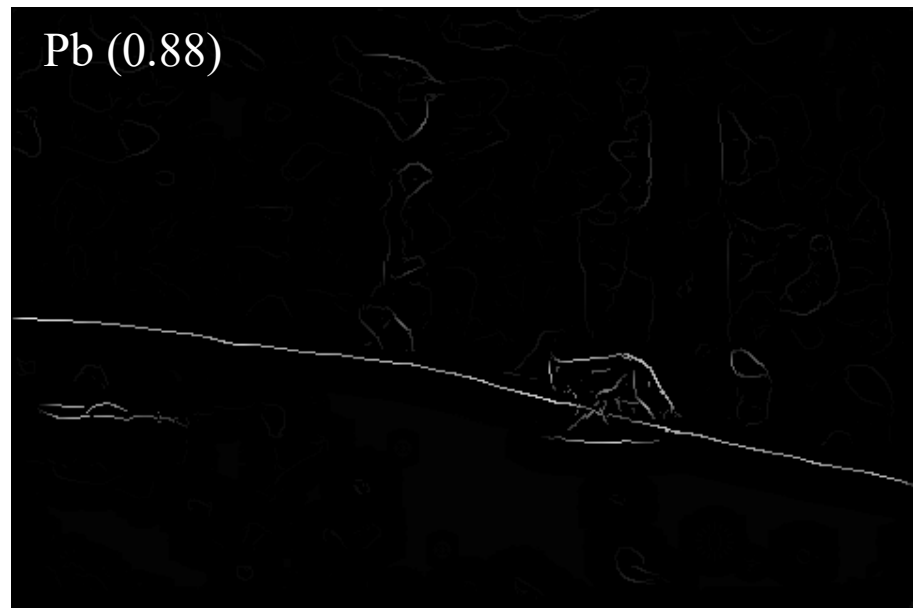
BG+CG+TG

Human



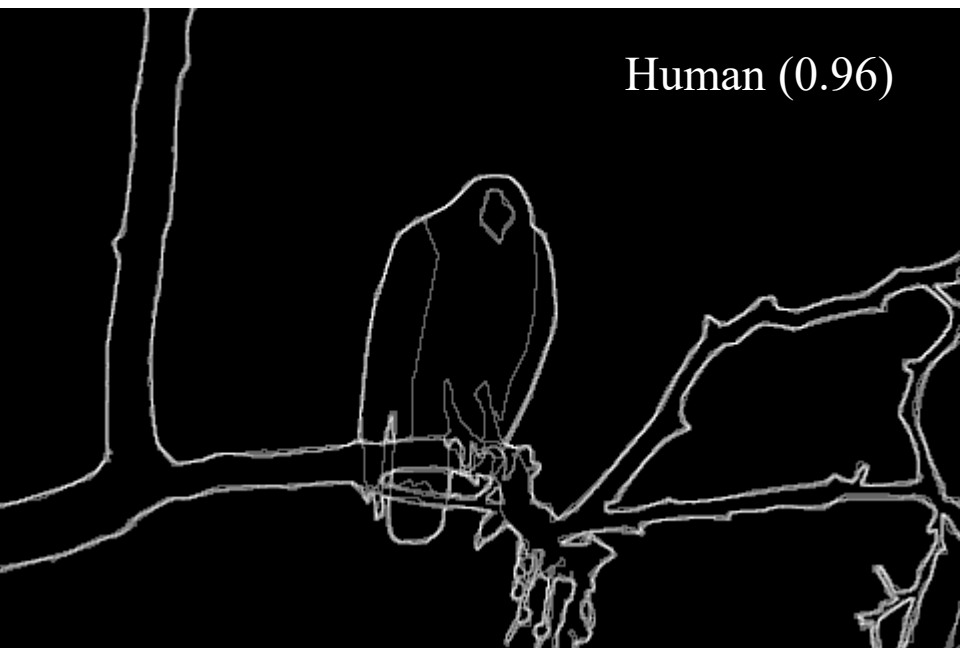


Results



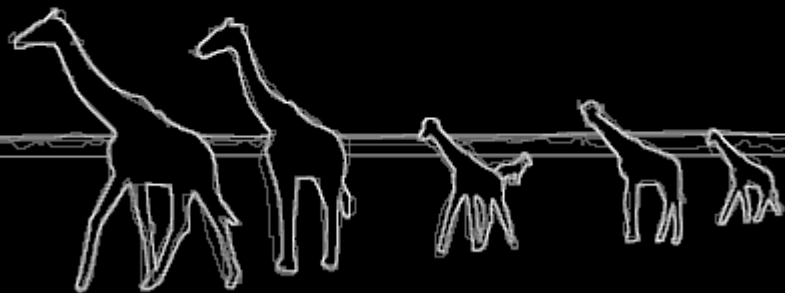


Results



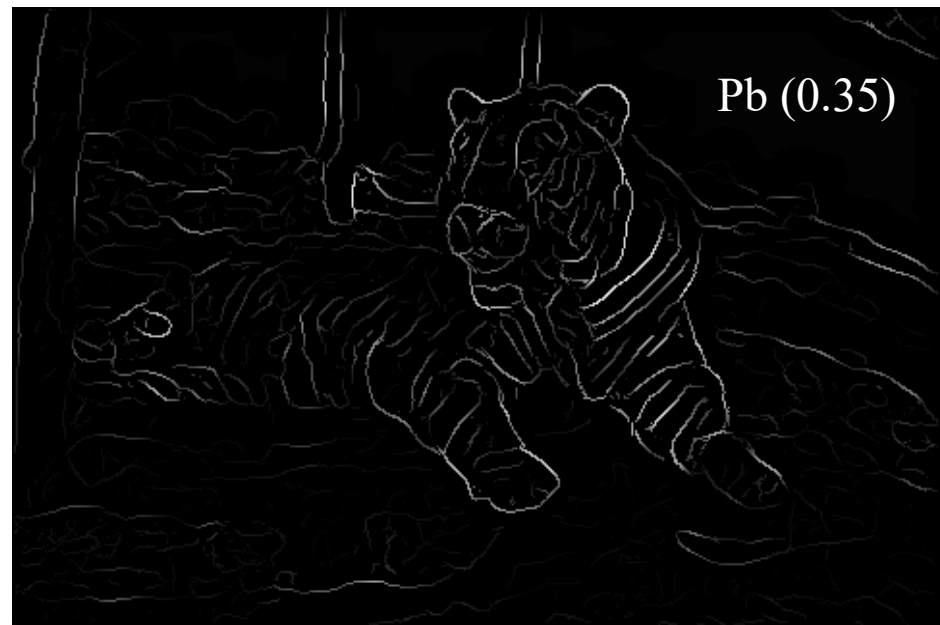


Human (0.95)



Pb (0.63)





For more:

<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/bench/html/108082-color.html>



Today's class



- Introduction to edge detection
- Gradients and edges
- Canny edge detector
- Object contour
- Pb edge detector
- **Straight line detection**



Finding straight lines





Finding line segments using connected components



1. Compute canny edges

- Compute: g_x, g_y (DoG in x,y directions)
- Compute: $\theta = \text{atan}(g_y / g_x)$

2. Assign each edge to one of 8 directions

3. For each direction d, get edgelets:

- find connected components for edge pixels with directions in $\{d-1, d, d+1\}$

4. Compute straightness and theta of edgelets using eig of x,y 2nd moment matrix of their points

$$\mathbf{M} = \begin{bmatrix} \sum (x - \mu_x)^2 & \sum (x - \mu_x)(y - \mu_y) \\ \sum (x - \mu_x)(y - \mu_y) & \sum (y - \mu_y)^2 \end{bmatrix} \quad [v, \lambda] = \text{eig}(\mathbf{M})$$

Larger eigenvector
↓
 $\theta = \text{atan2}(v(2,2), v(1,2))$
 $\text{conf} = \lambda_2 / \lambda_1$

5. Threshold on straightness, store segment



Canny lines $\rightarrow$... $\rightarrow$ straight edges





Things to remember

- Canny edge detector = smooth → derivative → thin → threshold → link
- Pb: learns weighting of gradient, color, texture differences
 - More recent learning approaches give at least as good accuracy and are faster
- Straight line detector = canny + gradient orientations → orientation binning → linking → check for straightness

